



OGC INDOORGML 2.0 PART 2C – SQL ENCODING

STANDARD
Implementation

CANDIDATE SWG DRAFT

Version: 0.5.0

Submission Date: 2026-06-18

Approval Date: 2026-XX-XX

Publication Date: 2026-XX-XX

Editor: Sisi Zlatanova, Zhiyong Wang, Taehoon Kim, Abdoulaye Diakite, Ki-Joune Li

Notice for Drafts: This document is not an OGC Standard. This document is distributed for review and comment. This document is subject to change without notice and may not be referred to as an OGC Standard.

Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

License Agreement

Use of this document is subject to the license agreement at <https://www.ogc.org/license>

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: <http://ogc.standardstracker.org/>

Copyright notice

Copyright © 2026 Open Geospatial Consortium

To obtain additional rights of use, visit <https://www.ogc.org/legal>

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. ABSTRACT	v
II. KEYWORDS	v
III. PREFACE	vi
IV. SECURITY CONSIDERATIONS	vii
V. SUBMITTING ORGANIZATIONS	viii
VI. SUBMITTERS	viii
1. SCOPE	2
2. CONFORMANCE	4
3. NORMATIVE REFERENCES	6
4. TERMS AND DEFINITIONS	8
5. CONVENTIONS	11
5.1. Symbols (and abbreviated terms)	11
5.2. Identifiers	11
5.3. Use of SQL terminology	11
5.4. Geometry storage	12
5.5. Complex DataType storage	13
5.6. Code list storage	13
5.7. Graph and route associations	14
5.8. Table and column naming	14
6. OVERVIEW OF THE SQL ENCODING	17
6.1. Conceptual model preservation	17
6.2. UML-to-SQL generation workflow	18
7. SQL ENCODING OF THE CORE MODULE	21
7.1. General requirements for encoding standard	21
7.2. IndoorFeatures	22
7.3. ThematicLayer	24
7.4. PrimalSpaceLayer	25
7.5. CellSpace	26

7.6. CellBoundary	28
7.7. DualSpaceLayer	30
7.8. Node	31
7.9. Edge	32
7.10. InterLayerConnection	34
7.11. ExternalReferenceType	36
8. SQL ENCODING OF THE NAVIGATION MODULE	39
8.1. General requirements for encoding standards	39
8.2. NavigableSpace	40
8.3. GeneralSpace	40
8.4. TransferSpace	41
8.5. NonNavigableSpace	42
8.6. ObjectSpace	42
8.7. NonNavigableBoundary	43
8.8. NavigableBoundary	43
8.9. Route	44
ANNEX A CONFORMANCE CLASS ABSTRACT TEST SUITE	47
A.1. Conformance Class Core	47
A.2. Conformance Class Navigation	51
ANNEX B SCHEMAS	56
B.1. SQL Schema of IndoorGML Core	56
B.2. SQL Schema of IndoorGML Navigation	63
ANNEX C CORE CODE LIST TYPES	69
ANNEX D NAVIGATION CODE LIST TYPES	71
ANNEX E EXAMPLE OF CODELIST	74
E.1. GeneralSpace	74
E.2. TransferSpace	79
ANNEX F REVISION HISTORY	81
BIBLIOGRAPHY	83



ABSTRACT

The IndoorGML 2.0 Part 2c – SQL Encoding Standard presents the implementation-dependent Structured Query Language (SQL) encoding of the concepts defined by the IndoorGML 2.0 Part 1 – Conceptual Model Standard (OGC 22-045r5). The SQL Encoding is compliant with SQL as specified by ISO/IEC 9075 and is implemented using the object-relational database PostgreSQL as the reference database management system. The SQL encoding of IndoorGML 2.0 defines a concrete implementation schema for representing indoor spatial information, including primal space, dual space, thematic layers, interlayer connections, and navigation-related indoor features. This encoding can be used to store and maintain 2D and/or 3D indoor space and indoor navigation network models in object-relational databases and can be accessed via an Application Programming Interface (API) or via web services.

This Part 2c of the Standard defines how the concepts developed in Part 1 are realized using SQL table definitions. The SQL encoding represents a mapping of the Conceptual Model derived from the UML model using Enterprise Architecture tooling, with subsequent corrections to identifier types, relationships, and missing properties. Geometry values are stored in PostGIS geometry columns. Geometry DataTypes from the conceptual model are flattened onto owning feature tables using path-style column names (for example, cellSpaceGeom_Geometry2D), and non-geometric reference DataTypes are encoded as PostgreSQL composite types. Table 1 maps requirements classes from the IndoorGML 2.0 Conceptual Model into the implementation details documented in this standard.

Table 1

CONCEPTUAL MODEL	SECTION	SQL SCHEMA
IndoorGML Core	Clause 7	Annex B.1
Navigation Extension	Clause 8	Annex B.2



KEYWORDS

The following are keywords to be used by search engines and document catalogues.

OGCDoc, OGC documents, indoor, navigation, IndoorGML, SQL, PostgreSQL, PostGIS



PREFACE

In order to achieve consensus on the modeling indoor spaces and their neighborhood relationships to support indoor location-based services, a UML Conceptual Model, IndoorGML 2.0 Part 1, was approved as an OGC Standard (OGC 22-045r5) in June, 2025. This model covers geometric and semantic properties of indoor spaces relevant for indoor navigation. This IndoorGML 2.0 Part 2c: SQL Encoding Standard defines how those concepts should be realized using a relational database schema. The SQL encoding of IndoorGML 2.0 is intended for software developers, data providers, standards implementers, and service providers who need a persistent, queryable, and interoperable representation of IndoorGML datasets in SQL databases.



SECURITY CONSIDERATIONS

No security considerations have been made for this Standard.

V

SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- University of New South Wales
- SpatialAlgebra Technology Limited
- Pusan National University
- CityGeometrix Pty Ltd
- National Institute of Advanced Industrial Science and Technology

VI

SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters:

NAME	AFFILIATION	EMAIL	OGC MEMBER
Sisi Zlatanova	University of New South Wales	s.zlatanova@unsw.edu.au	Yes
Zhiyong Wang	SpatialAlgebra Technology Limited	zhiyongwang@spatialalgebra.tech	Yes
Taehoon Kim	National Institute of Advanced Industrial Science and Technology	kim.taehoon@aist.go.jp	Yes
Abdoulaye Diakite	CityGeometrix Pty Ltd	abdou@citygeometrix.com	Yes
Ki-Joune Li	Pusan National University	lik@pnu.edu	Yes
Ruihang Xie	University of New South Wales	ruihang.xie@unsw.edu.au	No



1

SCOPE

The OGC IndoorGML 2.0 Part 2c – SQL Encoding Standard defines a SQL encoding for IndoorGML 2.0 Part 1 – Conceptual Model.

This Standard specifies:

- SQL table definitions for IndoorGML 2.0 core module;
- SQL table definitions for IndoorGML 2.0 navigation module;
- geometry storage rules for spatially enabled SQL database platforms, using PostgreSQL with PostGIS as the reference implementation; and
- a statement to which SQL encoding conformance classes an IndoorGML 2.0 implementation conforms to.

The SQL encoding of IndoorGML 2.0 is an implementation encoding of IndoorGML 2.0 Part 1. Therefore, database instances of the SQL encoding shall be interpreted according to the semantics defined in the IndoorGML 2.0 conceptual model.



2

CONFORMANCE

This standard defines an implementation specification which specifies how the IndoorGML 2.0 Conceptual Model should be implemented using Structured Query Language (SQL). The Standardization Target for this standard is:

- Implementations of the IndoorGML 2.0 Conceptual Model using SQL encodings.

Conformance with this standard shall be checked using all the relevant tests specified in Annex A (normative) of this document. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

In order to conform to this OGC® interface standard, a software implementation shall choose to implement:

- Any one of the conformance levels specified in Annex A (normative).

All requirements-classes and conformance-classes described in this document are owned by the standard(s) identified.

Table 2

CONFORMANCE CLASS	URI
IndoorGML Core	http://www.opengis.net/spec/indoorgml-2/2.0/sql/conf/core
Navigation Extension	http://www.opengis.net/spec/indoorgml-2/2.0/sql/conf/navi



3

NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Sisi Zlatanova, Abdoulaye Diakite, Taehoon Kim, Ki-Joune Li, Open Geospatial Consortium: OGC 22-045r5, *OGC IndoorGML 2.0 Part 1 – Conceptual Model*. Open Geospatial Consortium (2025). <http://www.opengis.net/doc/IS/indoorgml/2.0>.

International Organization for Standardization (committee): ISO 19101-1, *Geographic information – Reference model – Part 1: Fundamentals*. International Organization for Standardization, Geneva <https://www.iso.org/standard/59164.html>.

International Organization for Standardization (committee): ISO 19103, *Geographic information – Conceptual schema language*. International Organization for Standardization, Geneva <https://www.iso.org/standard/83454.html>.

International Organization for Standardization (committee): ISO 19107, *Geographic information – Spatial schema*. International Organization for Standardization, Geneva <https://www.iso.org/standard/66175.html>.

International Organization for Standardization (committee): ISO/IEC 9075, *Information technology – Database languages – SQL*. International Organization for Standardization, International Electrotechnical Commission, Geneva <https://www.iso.org/standard/16663.html>.

PostGIS Spatial and Geographic Objects for PostgreSQL, <https://postgis.net/docs/>



4

TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. conceptual model

model that defines concepts of a universe of discourse

[SOURCE: ISO 19101-1]

4.2. conceptual schema

formal description of a conceptual model

[SOURCE: ISO 19101-1]

4.3. data type

set of values that lack identity

Note 1 to entry: A data type is a class with no operations whose primary purpose is to hold information. In the IndoorGML 2.0 Conceptual Model, data types are denoted by the stereotype «DataType».

Note 2 to entry: In this SQL encoding, non-geometric UML DataTypes are encoded as PostgreSQL composite types, and geometry DataTypes are flattened onto owning feature tables as PostGIS geometry columns (see Clause 5).

[SOURCE: ISO 19103]

4.4. feature

abstraction of real world phenomena

Note 1 to entry: Features are objects that have an identity that allows for distinguishing features from each other. Features can have spatial and non-spatial properties that describe the features in more detail. Spatial properties are properties that have a geometry from ISO 19107 as data type.

Note 2 to entry: Features are denoted by the stereotype «FeatureType» in the IndoorGML 2.0 Conceptual Model.

[SOURCE: ISO 19101-1]

4.5. relational table

persistent data structure in a relational database that stores records of a single feature type or association

Note 1 to entry: In the SQL encoding of IndoorGML 2.0, each IndoorGML 2.0 feature class is mapped to one or more relational tables.



5

CONVENTIONS

5.1. Symbols (and abbreviated terms)

The following symbols and abbreviated terms are used in this standard. Symbols and abbreviated terms defined in IndoorGML 2.0 Part 1 also apply to this document.

Table 3 — Symbols and abbreviated terms

ABBREVIATION	WORD OR PHRASE
DDL	Data Definition Language
EA	Enterprise Architecture
JSON	JavaScript Object Notation
PostgreSQL	Open-source relational database management system
PostGIS	Spatial database extension for PostgreSQL
SQL	Structured Query Language

5.2. Identifiers

The normative provisions in this standard are denoted by the URI

<http://www.opengis.net/spec/indoorgml-2/2.0/sql>

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.3. Use of SQL terminology

The following terms are used as specified in SQL. In particular:

- “table” refers to a named relation in the database schema;
- “column” refers to an attribute of a table; and
- “row” refers to a single record in a table.

Feature identifiers are stored as string values in varchar columns rather than integer auto-increment keys, in order to preserve cross-references between IndoorGML objects.

5.4. Geometry storage

UML geometry DataTypes are flattened onto the owning feature table as PostGIS geometry columns (PostGIS) whose names follow the UML property path. The reference DDL and examples in this document use PostgreSQL with PostGIS for geometry storage.

- `CellSpace.cellSpaceGeom_Geometry2D` and `CellSpace.cellSpaceGeom_Geometry3D` (corresponding to `CellSpaceGeometryType`);
- `CellBoundary.cellBoundaryGeom_Geometry1D` and `CellBoundary.cellBoundaryGeom_Geometry2D` (corresponding to `CellBoundaryGeometryType`);
- `Node.geometry` and `Edge.geometry` (corresponding to the geometry property of `Node` and `Edge` in the conceptual model).

Although a PostGIS geometry column can hold any geometry type, each column of this encoding is restricted to the geometry types that correspond to its UML geometry class. The permitted values of the PostGIS `GeometryType` function are as follows.

Table 4 – Permitted geometry types per column

COLUMN	UML GEOMETRY CLASS	PERMITTED <code>GeometryType</code> VALUES
<code>CellSpace.cellSpaceGeom_Geometry2D</code>	<code>GM_Surface</code>	<code>POLYGON</code> , <code>MULTIPOLYGON</code>
<code>CellSpace.cellSpaceGeom_Geometry3D</code>	<code>GM_Solid</code>	<code>POLYHEDRALSURFACE</code> , <code>TIN</code>
<code>CellBoundary.cellBoundaryGeom_Geometry1D</code>	<code>GM_Curve</code>	<code>LINestring</code> , <code>MULTILINestring</code>

COLUMN	UML GEOMETRY CLASS	PERMITTED GeometryType VALUES
CellBoundary.cellBoundaryGeom_Geometry2D	GM_Surface	POLYGON, MULTIPOLYGON
Node.geometry	GM_Point	POINT
Edge.geometry	GM_Curve	LINESTRING, MULTILINESTRING

These restrictions are enforced by table CHECK constraints in the DDL of Annex B and are stated normatively in the requirements of Clause 7.

NOTE 1: The Geometry1D, Geometry2D, and Geometry3D column-name suffixes denote the UML geometry class of the property, not the coordinate dimension of the stored value. The PostGIS GeometryType function does not distinguish coordinate dimension; for example, a POLYGON Z value has the geometry type POLYGON. Implementations that require planar geometry only MAY additionally constrain a column with ST_NDims(...) = 2.

NOTE 2: Implementations on other SQL platforms (for example, Oracle Spatial) MAY conform by preserving the same logical table structure, keys, associations, and cardinalities, and by mapping geometry columns and PostgreSQL-specific constructs (ENUM, composite types, jsonb) to equivalent platform-native types (for example, SDO_GEOMETRY).

NOTE 3: IndoorGML GM_Solid (UML CellSpaceGeometryType.Geometry3D / column cellSpaceGeom_Geometry3D) has no native solid primitive in PostGIS. It is therefore stored in a bare PostGIS geometry column as a closed boundary shell, typically POLYHEDRALSURFACE Z or TIN Z. A table CHECK constraint on CellSpace restricts non-null cellSpaceGeom_Geometry3D values to those geometry types, requires three coordinate dimensions (ST_NDims(...) = 3), and requires that the value is a solid after CG_MakeSolid(...), as tested by CG_IsSolid(CG_MakeSolid(...)). These SFCGAL functions are provided by the postgis_sfcgal extension (PostGIS 3.5.0 and later). Volume and solid-topology operations use the same CG_ functions (CG_Volume, CG_MakeSolid, CG_IsSolid). Other SQL platforms MAY map the same logical column to a native solid type where available (for example, Oracle SDO_GEOMETRY solid).

5.5. Complex DataType storage

Non-geometric UML DataTypes that are not feature classes are encoded as PostgreSQL composite types (CREATE TYPE ... AS (...)) rather than as standalone tables. In particular, ExternalObjectReferenceType and ExternalReferenceType are composite types, and CellSpace / CellBoundary store an optional externalReference column of type ExternalReferenceType.

5.6. Code list storage

IndoorGML code list types (for example, ThemeLayerValue, TopoExpressionValue, LocomotionAccessType) are represented as PostgreSQL ENUM types whose labels match the UML type names and enumeration literals defined in the IndoorGML 2.0 conceptual model.

5.7. Graph and route associations

Graph adjacency in the dual space layer is encoded as follows:

- Node.connects stores the identifiers of connected Edge rows as a jsonb array;
- Edge.connects_node1 and Edge.connects_node2 store the two connected Node identifiers as foreign-key columns referencing Node.NodeID.

A Route represents an ordered navigation path through the dual-space graph. The ordered routeNode and routeEdge members are stored as jsonb arrays on the Route table rather than in separate junction tables. This encoding preserves the sequence of nodes and edges in a single row and simplifies maintenance of route order during import, update, and export. Array position expresses traversal order; the first element is the first node or edge in the route, the second element is the next, and so on. Both routeNode and routeEdge SHALL be present and non-empty on every Route row.

Example:

```
-- routeNode and routeEdge preserve traversal order in array position
INSERT INTO "Route" ("RouteID", "routeNode", "routeEdge")
VALUES (
    'R1',
    '['N1", "N2", "N3"]'::jsonb,
    '['E1", "E2"]'::jsonb
);

-- Edge endpoints reference Node rows via foreign keys
INSERT INTO "Edge" ("EdgeID", "DualSpaceLayerID", "weight", connects_node1,
connects_node2)
VALUES ('E1', 'DS-1', 1.0, 'N1', 'N2');
```

Listing 1

5.8. Table and column naming

SQL table and column names use identifiers enclosed in double quotes. Table names use PascalCase aligned with UML type names (for example, IndoorFeatures, ThematicLayer, CellSpace, NavigableSpace, ObjectSpace).



6

OVERVIEW OF THE SQL ENCODING

The SQL encoding of IndoorGML 2.0 implements the conceptual classes of IndoorGML 2.0 Part 1 as relational database tables. The conceptual model remains normative for semantics, while this document defines the SQL representation.

The SQL encoding is based on the following encoding principles.

6.1. Conceptual model preservation

The SQL encoding of the IndoorGML core module preserves the IndoorGML 2.0 core conceptual model structure as close as possible, as shown in Figure 1.

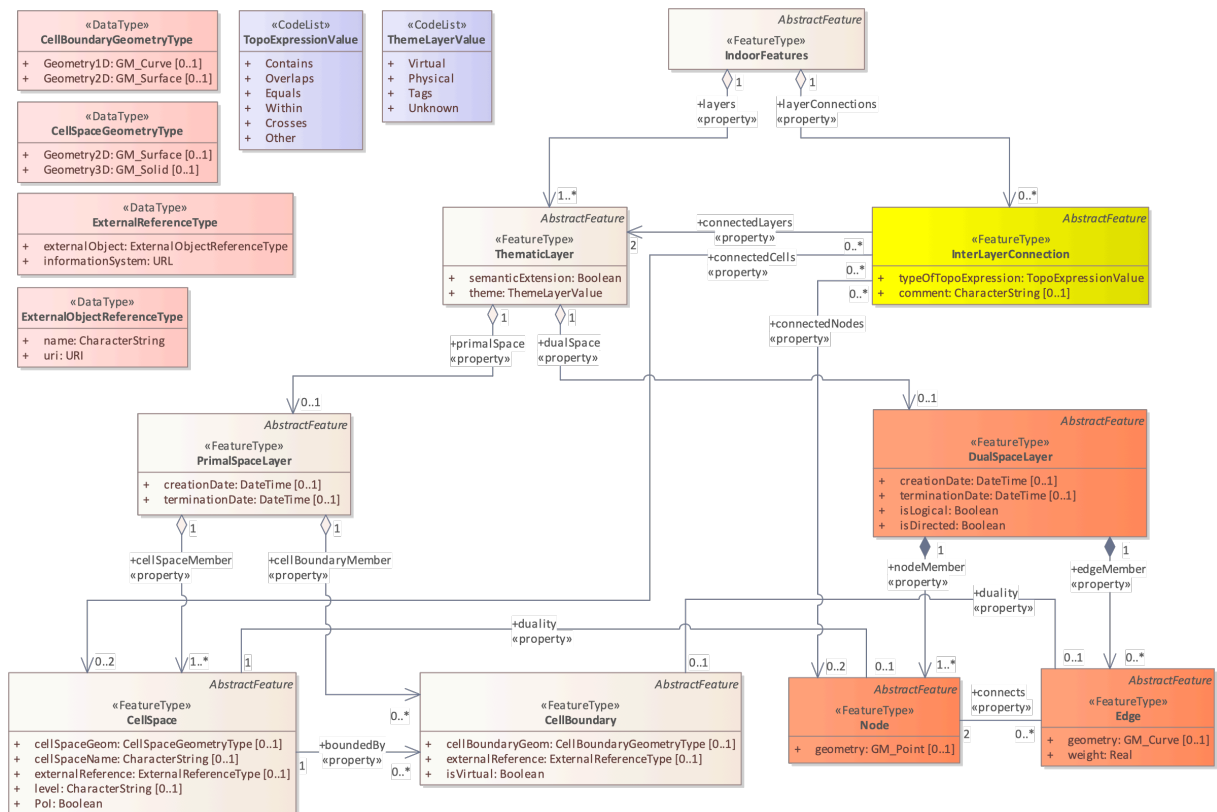


Figure 1 – IndoorGML 2.0 core module structure

Each feature class is mapped to one or more relational tables, and associations are represented using foreign key columns or junction tables. For example, the IndoorFeatures table is related to ThematicLayer rows through the IndoorFeaturesID foreign key, and an InterLayerConnection row references two thematic layers, and optionally a pair of nodes and a

pair of cell spaces, through the connectedLayers_1 / connectedLayers_2, connectedNodes_1 / connectedNodes_2, and connectedCells_1 / connectedCells_2 foreign-key columns.

The SQL encoding of the IndoorGML navigation module preserves the IndoorGML 2.0 navigation conceptual model structure as close as possible, as shown in Figure 2.

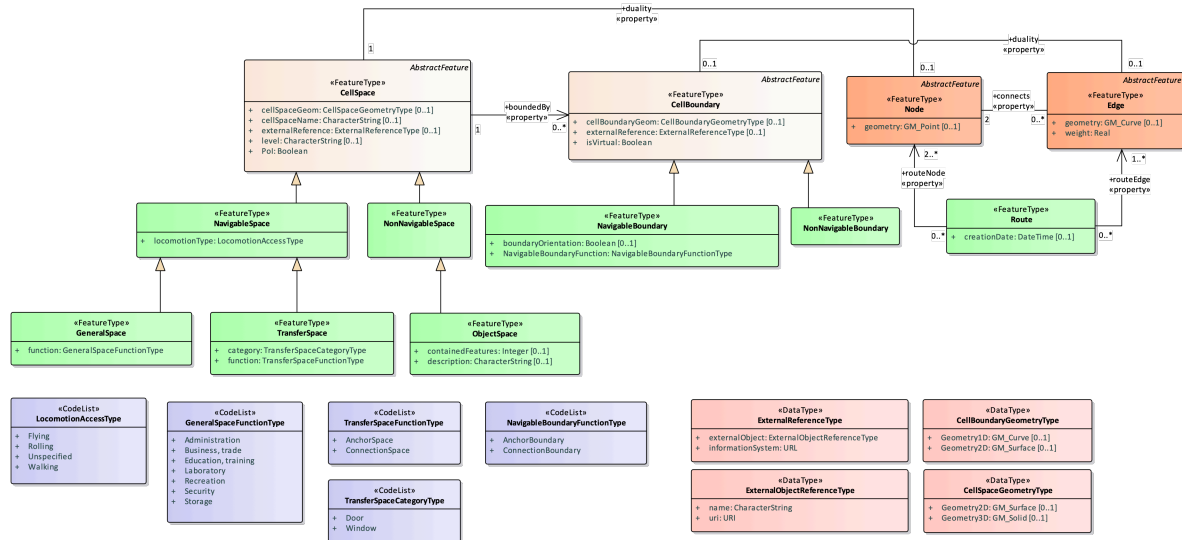


Figure 2 – IndoorGML 2.0 navigation module structure

Navigation feature classes extend core tables using table inheritance patterns implemented through foreign key references to CellSpace and CellBoundary.

6.2. UML-to-SQL generation workflow

The SQL encoding schema is derived from the IndoorGML 2.0 UML model and is designed on the basis of PostgreSQL with PostGIS as the reference database platform. SQL CREATE TABLE statements are first generated from the UML model using Enterprise Architecture tooling, and the resulting DDL is then corrected to align with the conceptual model. Typical corrections include:

- converting identifier properties from integer columns to string (varchar) columns;
- replacing EA code list tables with PostgreSQL ENUM types named after the UML code list types;
- flattening geometry DataType properties onto owning feature tables as path-named PostGIS columns (for example, cellSpaceGeom_Geometry2D / cellSpaceGeom_Geometry3D);
- replacing EA tables for ExternalReferenceType / ExternalObjectReferenceType with PostgreSQL composite types used as property columns;

- storing the `Node.connects` property as a jsonb array of connected Edge identifiers;
- storing the Edge endpoint properties as `connects_node1` / `connects_node2` foreign keys to Node;
- storing the `InterLayerConnection` associations as paired foreign keys (`connectedLayers_1` / `connectedLayers_2`, `connectedNodes_1` / `connectedNodes_2`, `connectedCells_1` / `connectedCells_2`);
- storing the `routeNode` and `routeEdge` properties as ordered jsonb arrays on Route to preserve traversal sequence;
- adding missing properties (for example, the `layers` property on `IndoorFeatures`);
- updating relationship cardinalities and junction tables.



7

SQL ENCODING OF THE CORE MODULE

REQUIREMENTS CLASS 1

IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
TARGET TYPE	Implementation Specification
CONFORMANCE CLASS	Conformance class A.1: http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/core
PREREQUISITE	http://www.opengis.net/spec/indoorgml/2.0/req
NORMATIVE STATEMENTS	Requirement 1: /req/core/global Requirement 2: /req/core/indoorsql Requirement 3: /req/core/indoorfeature Requirement 4: /req/core/thematiclayer Requirement 5: /req/core/primalspacelayer Requirement 6: /req/core/cellspace Requirement 7: /req/core/cellboundary Requirement 8: /req/core/dualspacelayer Requirement 9: /req/core/node Requirement 10: /req/core/edge Requirement 11: /req/core/interlayerconnection

7.1. General requirements for encoding standard

An implementation of IndoorGML 2.0 Core using SQL SHALL implement the relational schema defined by this standard.

REQUIREMENT 1: REQUIREMENT FOR GLOBAL SQL SCHEMA

IDENTIFIER	/req/core/global
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A database representing an IndoorGML 2.0 Core dataset SHALL implement the SQL schema defined in this standard (including IndoorGML_core.sql and the PostGIS geometry types specified in Clause 5).

Each feature table of the SQL encoding represents one feature class of the IndoorGML 2.0 conceptual model.

Every feature table:

- **must** have a primary key column storing a string identifier that serves as a unique identifier for the feature and can be used to represent cross-references between objects in the SQL encoding.

REQUIREMENT 2: REQUIREMENT FOR SQL ENCODING FEATURE TABLES	
IDENTIFIER	/req/core/indoorsql
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	Each feature table of the SQL encoding SHALL contain a string primary key column for the feature identifier.

Core code list ENUM types used by the core module tables are specified in Annex C.

7.2. IndoorFeatures

The IndoorFeatures table stores instances of IndoorFeature, the container for the indoor dataset. It aggregates one or more thematic layers and may include inter-layer connections.

The IndoorFeatures table shall contain:

- IndoorFeaturesID as the primary key
- optionally, layers as a reference to associated thematic layers

NOTE: The layers column was added during schema correction because it was not included in the SQL generated from Enterprise Architecture tooling.

The following examples refer to an IndoorFeatures row shown in Figure 3. In the example, the indoor dataset aggregates ThematicLayer rows linked through the IndoorFeaturesID foreign key and InterLayerConnection rows stored in the InterLayerConnection table. Each row is identified by a string primary key, which is referenced by foreign-key columns (such as connectedLayers_1 or connectedNodes_1 in InterLayerConnection).

REQUIREMENT 3: INDOORFEATURES SCHEMA

IDENTIFIER	/req/core/indoorfeature
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An IndoorFeatures table SHALL be related to at least one ThematicLayer row through a foreign key.

7.3. ThematicLayer

The ThematicLayer table stores instances of ThematicLayer, which represents one contextual decomposition of indoor space.

The ThematicLayer table shall contain:

- ThematicLayerID as the primary key
- IndoorFeaturesID as a foreign key to IndoorFeatures
- semanticExtension indicates whether there is an extension module with additional semantic information (mandatory)
- theme of type ThemeLayerValue, with values including "Virtual", "Physical", "Tags", and "Unknown" (mandatory)

```
CREATE TABLE "ThematicLayer"
(
    "semanticExtension" boolean NOT NULL,
    "theme" "ThemeLayerValue" NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NOT NULL,
    CONSTRAINT "PK_ThematicLayer" PRIMARY KEY ("ThematicLayerID"),
    CONSTRAINT "FK_ThematicLayer_layers"
        FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
        ("IndoorFeaturesID")
);

COMMENT ON TABLE "ThematicLayer"
    IS 'A layer of specific theme aggregating a primal and/or a dual space of a
    given environment.';

COMMENT ON COLUMN "ThematicLayer"."semanticExtension"
    IS 'Indicates whether semantic information is associated (true) or not
    (false) to the primal space of the thematic layer.';

COMMENT ON COLUMN "ThematicLayer"."theme"
```



```
IS 'Determines the theme of the layer (e.g topographic, logical, etc.).';
```

Listing 4 – A SQL schema of the ThematicLayer table

The following examples refer to a ThematicLayer row (“TL-Pedestrian”), shown in Figure 3.

```
INSERT INTO "ThematicLayer" (
  "ThematicLayerID", "IndoorFeaturesID", "semanticExtension", "theme"
) VALUES ('TL-Pedestrian', 'IFs', false, 'Physical');

-- PrimalSpaceLayer (PS-1) and DualSpaceLayer (DS-1) rows of this layer are
inserted separately.
```

Listing 5 – An example INSERT statement for the ThematicLayer table

REQUIREMENT 4: THEMATICLAYER SCHEMA

IDENTIFIER	/req/core/thematiclayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A ThematicLayer table SHALL declare whether there is an extension module with additional semantic information using the semanticExtension column.
B	A ThematicLayer table SHALL determine what type of model representation can be expected using the theme column.
C	The theme value SHALL be one of “Virtual”, “Physical”, “Tags”, and “Unknown”.

7.4. PrimalSpaceLayer

The PrimalSpaceLayer table stores instances of PrimalSpaceLayer, which contains spatial objects representing indoor subdivisions.

The PrimalSpaceLayer table shall contain:

- PrimalSpaceLayerID as the primary key
- ThematicLayerID as a foreign key to ThematicLayer (mandatory)
- optionally, creationDate
- optionally, terminationDate

```
CREATE TABLE "PrimalSpaceLayer"
(
  "creationDate" timestamp without time zone NULL,
  "terminationDate" timestamp without time zone NULL,
  "PrimalSpaceLayerID" varchar(100) NOT NULL,
```

```

    "ThematicLayerID" varchar(100) NOT NULL,
    CONSTRAINT "PK_PrimalSpaceLayer" PRIMARY KEY ("PrimalSpaceLayerID"),
    CONSTRAINT "FK_PrimalSpaceLayer_primalSpace"
        FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer"
        ("ThematicLayerID")
);

```

Listing 6 — A SQL schema of the PrimalSpaceLayer table

The following examples refer to a PrimalSpaceLayer row of “TL-Pedestrian”, shown in Figure 3.

```

INSERT INTO "PrimalSpaceLayer" ("PrimalSpaceLayerID", "ThematicLayerID",
"creationDate")
VALUES ('PS-1', 'TL-Pedestrian', '2025-10-30 13:00:00');

```

-- CellSpace and CellBoundary rows of this layer are inserted separately.

Listing 7 — An example INSERT statement for the PrimalSpaceLayer table

REQUIREMENT 5: PRIMALSPACE LAYER SCHEMA

IDENTIFIER	/req/core/primalspacelayer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A PrimalSpaceLayer table SHALL be related to one or more CellSpace rows.

7.5. CellSpace

The CellSpace table stores instances of CellSpace, which represents a unit of indoor space.

The CellSpace table shall contain:

- CellSpaceID as the primary key
- PrimalSpaceLayerID as a foreign key to PrimalSpaceLayer
- PoI is a flag that specifies whether this cell space is a point of interest (mandatory)
- duality referencing the NodeID of the dual node (mandatory)
- optionally, cellSpaceName
- optionally, level
- optionally, cellSpaceGeom_Geometry2D as a PostGIS geometry value (UML CellSpaceGeometryType.Geometry2D)

- optionally, cellSpaceGeom_Geometry3D as a PostGIS geometry value representing UML GM_Solid / CellSpaceGeometryType.Geometry3D as a closed boundary shell (POLYHEDRALSURFACE Z or TIN Z; see Clause 5)
- optionally, externalReference of composite type ExternalReferenceType

When cellSpaceGeom_Geometry3D is present, a CHECK constraint requires GeometryType(...) to be POLYHEDRALSURFACE or TIN, ST_NDims(...) to be 3, and CG_IsSolid(CG_MakeSolid(...)) to be true (the stored shell is converted to a solid and then tested; see Clause 5).

```
CREATE TABLE "CellSpace"
(
    "cellSpaceGeom_Geometry2D" geometry NULL,
    "cellSpaceGeom_Geometry3D" geometry NULL,
    "cellSpaceName" varchar(100) NULL,
    "externalReference" "ExternalReferenceType" NULL,
    "level" varchar(100) NULL,
    "PoI" boolean NOT NULL,
    "CellSpaceID" varchar(100) NOT NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "duality" varchar(100) NOT NULL,
    CONSTRAINT "PK_CellSpace" PRIMARY KEY ("CellSpaceID"),
    CONSTRAINT "chk_CellSpace_geom2d_surface" CHECK (
        "cellSpaceGeom_Geometry2D" IS NULL
        OR GeometryType("cellSpaceGeom_Geometry2D") IN ('POLYGON',
'MULTIPOLYGON')
    ),
    CONSTRAINT "chk_CellSpace_geom3d_solid" CHECK (
        "cellSpaceGeom_Geometry3D" IS NULL
        OR (
            GeometryType("cellSpaceGeom_Geometry3D") IN ('POLYHEDRALSURFACE',
'TIN')
            AND ST_NDims("cellSpaceGeom_Geometry3D") = 3
            AND CG_IsSolid(CG_MakeSolid("cellSpaceGeom_Geometry3D"))
        )
    ),
    CONSTRAINT "FK_CellSpace_cellSpaceMember"
        FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID")
);
```

Listing 8 – A SQL schema of the CellSpace table

The following examples refer to a CellSpace row ("R1"), shown in Figure 3.

```
INSERT INTO "CellSpace" (
    "CellSpaceID", "PrimalSpaceLayerID", "level", "PoI",
    "cellSpaceGeom_Geometry2D", "duality"
) VALUES (
    'R1', 'PS-1', '1', false,
    ST_SetSRID(ST_GeomFromGeoJSON(
        '{"type": "Polygon", "coordinates": [[[0,0.25],[3.25,0.25],[3.25,1.5],
[0,1.5],[0,0.25]]]}'
    ), 0),
    'N-R1'
);
```

-- CellBoundary rows B1, B2, B7, and B8 reference this cell space through boundedBy.

Listing 9 – An example INSERT statement for the CellSpace table

REQUIREMENT 6: CELLSPACE SCHEMA

IDENTIFIER /req/core/cellspace

INCLUDED IN Requirements class 1: <http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/core>

A A CellSpace table SHALL declare whether it is a point of interest space using the PoI column.

B A CellSpace table SHALL contain a duality value referencing the NodeID of the corresponding Node.

C A non-null cellSpaceGeom_Geometry2D value SHALL have the geometry type POLYGON or MULTIPOLYGON.

D A non-null cellSpaceGeom_Geometry3D value SHALL have the geometry type POLYHEDRALSURFACE or TIN.

E A non-null cellSpaceGeom_Geometry3D value SHALL have three coordinate dimensions (ST_NDims(...) = 3) and SHALL be a solid after CG_MakeSolid(...), as tested by CG_IsSolid(CG_MakeSolid(...)).

7.6. CellBoundary

The CellBoundary table stores instances of CellBoundary, a boundary of a CellSpace. CellBoundary rows reference CellSpace rows through the boundedBy foreign key column. Optionally, a CellBoundary may be the dual of an Edge; this association is encoded on the Edge table via the optional Edge.duality foreign key referencing CellBoundaryID.

The CellBoundary table shall contain:

- CellBoundaryID as the primary key
- PrimalSpaceLayerID as a foreign key to PrimalSpaceLayer (mandatory)
- isVirtual indicating whether the boundary is virtual or physical (mandatory)
- optionally, boundedBy referencing the CellSpaceID of a bounded cell space
- optionally, cellBoundaryGeom_Geometry1D as a PostGIS geometry value (UML CellBoundaryGeometryType.Geometry1D, typically a curve)

- optionally, cellBoundaryGeom_Geometry2D as a PostGIS geometry value (UML CellBoundaryGeometryType.Geometry2D, typically a surface)
- optionally, externalReference of composite type ExternalReferenceType

```
CREATE TABLE "CellBoundary"
(
    "cellBoundaryGeom_Geometry1D" geometry NULL,
    "cellBoundaryGeom_Geometry2D" geometry NULL,
    "externalReference" "ExternalReferenceType" NULL,
    "isVirtual" boolean NOT NULL,
    "CellBoundaryID" varchar(100) NOT NULL,
    "boundedBy" varchar(100) NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    CONSTRAINT "PK_CellBoundary" PRIMARY KEY ("CellBoundaryID"),
    CONSTRAINT "chk_CellBoundary_geom1d_curve" CHECK (
        "cellBoundaryGeom_Geometry1D" IS NULL
        OR GeometryType("cellBoundaryGeom_Geometry1D") IN ('LINESTRING',
'MULTILINESTRING')
    ),
    CONSTRAINT "chk_CellBoundary_geom2d_surface" CHECK (
        "cellBoundaryGeom_Geometry2D" IS NULL
        OR GeometryType("cellBoundaryGeom_Geometry2D") IN ('POLYGON',
'MULTIPOLYGON')
    ),
    CONSTRAINT "FK_CellBoundary_boundedBy"
        FOREIGN KEY ("boundedBy") REFERENCES "CellSpace" ("CellSpaceID"),
    CONSTRAINT "FK_CellBoundary_cellBoundaryMember"
        FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID")
);
```

Listing 10 – A SQL schema of the CellBoundary table

The following examples refer to a CellBoundary row ("B1"), shown in Figure 3.

```
INSERT INTO "CellBoundary" (
    "CellBoundaryID", "PrimalSpaceLayerID", "isVirtual",
    "cellBoundaryGeom_Geometry1D", "boundedBy"
) VALUES (
    'B1', 'PS-1', false,
    ST_SetSRID(ST_GeomFromGeoJSON(
        '{"type":"LineString","coordinates":[[0.5,0.25],[1.0,0.25]]}'
    ), 0),
    'R1'
);

-- The dual Edge (E1) references this cell boundary through Edge.duality.
```

Listing 11 – An example INSERT statement for the CellBoundary table

REQUIREMENT 7: CELLBOUNDARY SCHEMA

IDENTIFIER /req/core/cellboundary

INCLUDED IN Requirements class 1: <http://www.opengis.net/spec/indoorqml-2/2.0/sql/req/core>

REQUIREMENT 7: CELLBOUNDARY SCHEMA

A	A CellBoundary table SHALL specify whether it is virtual using the isVirtual column.
B	A non-null cellBoundaryGeom_Geometry1D value SHALL have the geometry type LINESTRING or MULTILINESTRING.
C	A non-null cellBoundaryGeom_Geometry2D value SHALL have the geometry type POLYGON or MULTIPOLYGON.

7.7. DualSpaceLayer

The DualSpaceLayer table stores instances of DualSpaceLayer, representing the graph structure corresponding to the primal space layer.

The DualSpaceLayer table shall contain:

- DualSpaceLayerID as the primary key
- ThematicLayerID as a foreign key to ThematicLayer (mandatory)
- isLogical indicating whether the graph is geometric or logical (mandatory)
- isDirected specifying whether the graph is directed (mandatory)
- optionally, creationDate
- optionally, terminationDate

```
CREATE TABLE "DualSpaceLayer"
(
    "creationDate" timestamp without time zone NULL,
    "terminationDate" timestamp without time zone NULL,
    "isLogical" boolean NOT NULL,
    "isDirected" boolean NOT NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    CONSTRAINT "PK_DualSpaceLayer" PRIMARY KEY ("DualSpaceLayerID"),
    CONSTRAINT "FK_DualSpaceLayer_dualSpace"
        FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer"
        ("ThematicLayerID")
);
```

Listing 12 – A SQL schema of the DualSpaceLayer table

The following examples refer to a DualSpaceLayer row of “TL-Pedestrian”, shown in Figure 3.

```
INSERT INTO "DualSpaceLayer" (
    "DualSpaceLayerID", "ThematicLayerID", "creationDate", "isLogical",
    "isDirected"
) VALUES ('DS-1', 'TL-Pedestrian', '2025-10-30 13:00:00', false, false);
```

-- Node and Edge rows of this layer are inserted separately.

Listing 13 – An example INSERT statement for the DualSpaceLayer table

REQUIREMENT 8: DUALSPACE LAYER SCHEMA

IDENTIFIER	/req/core/dualspace layer
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A DualSpaceLayer table SHALL specify whether the provided graph is geometric or logical using the isLogical column.
B	A DualSpaceLayer table SHALL specify whether the provided graph is directed using the isDirected column.
C	A DualSpaceLayer table SHALL be related to one or more Node rows.

7.8. Node

The Node table stores instances of Node, a state or graph vertex in the dual space layer.

The Node table shall contain:

- NodeID as the primary key
- DualSpaceLayerID as a foreign key to DualSpaceLayer (mandatory)
- duality referencing the CellSpaceID of the corresponding cell space (mandatory)
- optionally, geometry as a PostGIS geometry value (typically Point)
- optionally, connects as a jsonb array of connected EdgeID values

```
CREATE TABLE "Node"
(
    "geometry" geometry NULL,
    "NodeID" varchar(100) NOT NULL,
    "duality" varchar(100) NOT NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    connects jsonb NULL,
    CONSTRAINT "PK_Node" PRIMARY KEY ("NodeID"),
    CONSTRAINT "chk_Node_geometry_point" CHECK (
        "geometry" IS NULL
        OR GeometryType("geometry") = 'POINT'
    ),
    CONSTRAINT "FK_Node_duality"
```

```

        FOREIGN KEY ("duality") REFERENCES "CellSpace" ("CellSpaceID"),
        CONSTRAINT "FK_Node_DualSpaceLayer"
        FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
        ("DualSpaceLayerID")
    );

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_duality"
    FOREIGN KEY ("duality") REFERENCES "Node" ("NodeID");

```

Listing 14 – A SQL schema of the Node table

The following examples refer to a Node row ("N-R1"), shown in Figure 3.

```

INSERT INTO "Node" ("NodeID", "DualSpaceLayerID", "duality", "geometry",
connects)
VALUES (
    'N-R1', 'DS-1', 'R1',
    ST_SetSRID(ST_GeomFromGeoJSON('{"type":"Point","coordinates":
[1.625,0.875]}'), 0),
    '["E1","E2","E7","E8"]'::jsonb
);

```

Listing 15 – An example INSERT statement for the Node table

REQUIREMENT 9: NODE SCHEMA

IDENTIFIER	/req/core/node
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	A Node table SHALL contain a duality value referencing the CellSpaceID of the corresponding CellSpace.
B	A Node table SHALL store connected edge identifiers in the connects column.
C	A non-null Node geometry value SHALL have the geometry type POINT.

7.9. Edge

The Edge table stores instances of Edge, representing a relationship between two nodes.

The Edge table shall contain:

- EdgeID as the primary key
- DualSpaceLayerID as a foreign key to DualSpaceLayer (mandatory)
- weight for use in graph-based applications (mandatory)

- connects_node1 and connects_node2 as foreign keys to the two connected Node rows (mandatory)
- optionally, duality referencing the CellBoundaryID of the corresponding cell boundary
- optionally, geometry as a PostGIS geometry value (typically LineString)

NOTE: If no list of connected edges is presented on Node (that is, Node.connects is absent or empty), the Edge table SHALL provide the connect properties connects_node1 and connects_node2 so that the dual-space graph adjacency can be determined from the Edge rows.

```
CREATE TABLE "Edge"
(
    "geometry" geometry NULL,
    "weight" real NOT NULL,
    "EdgeID" varchar(100) NOT NULL,
    "duality" varchar(100) NULL,
    "DualSpaceLayerID" varchar(100) NOT NULL,
    connects_node1 varchar(100) NOT NULL,
    connects_node2 varchar(100) NOT NULL,
    CONSTRAINT "PK_Edge" PRIMARY KEY ("EdgeID"),
    CONSTRAINT "chk_Edge_geometry_curve" CHECK (
        "geometry" IS NULL
        OR GeometryType("geometry") IN ('LINESTRING', 'MULTILINESTRING')
    ),
    CONSTRAINT "FK_Edge_duality"
        FOREIGN KEY ("duality") REFERENCES "CellBoundary" ("CellBoundaryID"),
    CONSTRAINT "FK_Edge_DualSpaceLayer"
        FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
        ("DualSpaceLayerID"),
    CONSTRAINT "FK_Edge_connects_node1"
        FOREIGN KEY (connects_node1) REFERENCES "Node" ("NodeID"),
    CONSTRAINT "FK_Edge_connects_node2"
        FOREIGN KEY (connects_node2) REFERENCES "Node" ("NodeID")
);
```

Listing 16 – A SQL schema of the Edge table

The following examples refer to an Edge row ("E1"), shown in Figure 3.

```
INSERT INTO "Edge" (
    "EdgeID", "DualSpaceLayerID", "duality", "weight", "geometry",
    "connects_node1", "connects_node2"
) VALUES (
    'E1', 'DS-1', 'B1', 1.0,
    ST_SetSRID(ST_GeomFromGeoJSON(
        '{"type": "LineString", "coordinates": [[0.75, 0.125], [0.75, 0.875],
        [1.625, 0.875]]}'
    ), 0),
    'N-D1', 'N-R1'
);
```

Listing 17 – An example INSERT statement for the Edge table

REQUIREMENT 10: EDGE SCHEMA

IDENTIFIER /req/core/edge

REQUIREMENT 10: EDGE SCHEMA

INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An Edge table SHALL reference connected Nodes using the connects_node1 and connects_node2 columns.
B	An Edge table SHALL contain a weight column.
C	A non-null Edge geometry value SHALL have the geometry type LINESTRING or MULTILINESTRING.

7.10. InterLayerConnection

The `InterLayerConnection` table stores instances of `InterLayerConnection`. In the IndoorGML 2.0 conceptual model, an `InterLayerConnection` describes the connection between two thematic layers and may also identify the cell spaces and/or nodes that participate in that connection. It is used when objects in different thematic layers (for example, a pedestrian layer and a wheelchair layer, or a room layer and a furniture layer) need an explicit topological relationship.

The `InterLayerConnection` table shall contain:

- `InterLayerConnectionID` as the primary key
- `IndoorFeaturesID` as a foreign key to `IndoorFeatures`
- `typeOfTopoExpression` of type `TopoExpressionValue`, with values including “Contains”, “Overlaps”, “Equals”, “Within”, “Crosses”, and “Other” (mandatory)
- `connectedLayers_1` and `connectedLayers_2` as foreign keys to the two connected `ThematicLayer` rows (mandatory)
- optionally, `connectedNodes_1` and `connectedNodes_2` as foreign keys to a pair of connected `Node` rows
- optionally, `connectedCells_1` and `connectedCells_2` as foreign keys to a pair of connected `CellSpace` rows
- optionally, `comment`

The `typeOfTopoExpression` column represents the topological relationship between the two layers. Its values include “Contains”, “Overlaps”, “Equals”, “Within”, “Crosses”, and “Other”, according to the conceptual model. Those topological values are verbs for which the subject is the first connected layer: one shall read `connectedLayers_1 typeOfTopoExpression connectedLayers_2` (for example, Layer Room *Contains* Layer Furniture).

The paired columns encode the ordered associations of the conceptual model and SHALL be mapped to the same indices. connectedLayers_1, connectedNodes_1, and connectedCells_1 refer to the first side of the relationship (the subject of typeOfTopoExpression); connectedLayers_2, connectedNodes_2, and connectedCells_2 refer to the second side. For example, if typeOfTopoExpression is “Contains”, then connectedCells_1 contains connectedCells_2 (and likewise for the corresponding nodes, when present). When present, connectedNodes_* and connectedCells_* shall be populated as a pair (both null, or both non-null), corresponding to the IndoorGML cardinalities of 0 or 2. connectedCells is used when the connection is between primal spaces; connectedNodes is used when the connection is between dual spaces.

```
CREATE TABLE "InterLayerConnection"
(
    "typeOfTopoExpression" "TopoExpressionValue" NOT NULL,
    "comment" text NULL,
    "InterLayerConnectionID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NULL,
    "connectedLayers_1" varchar(100) NOT NULL,
    "connectedLayers_2" varchar(100) NOT NULL,
    "connectedNodes_1" varchar(100) NULL,
    "connectedNodes_2" varchar(100) NULL,
    "connectedCells_1" varchar(100) NULL,
    "connectedCells_2" varchar(100) NULL,
    CONSTRAINT "PK_InterLayerConnection" PRIMARY KEY ("InterLayerConnectionID"),
    CONSTRAINT "FK_InterLayerConnection_layerConnections"
        FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
("IndoorFeaturesID"),
    CONSTRAINT "FK_InterLayerConnection_connectedLayers_1"
        FOREIGN KEY ("connectedLayers_1") REFERENCES "ThematicLayer"
("ThematicLayerID"),
    CONSTRAINT "FK_InterLayerConnection_connectedLayers_2"
        FOREIGN KEY ("connectedLayers_2") REFERENCES "ThematicLayer"
("ThematicLayerID"),
    CONSTRAINT "FK_InterLayerConnection_connectedNodes_1"
        FOREIGN KEY ("connectedNodes_1") REFERENCES "Node" ("NodeID"),
    CONSTRAINT "FK_InterLayerConnection_connectedNodes_2"
        FOREIGN KEY ("connectedNodes_2") REFERENCES "Node" ("NodeID"),
    CONSTRAINT "FK_InterLayerConnection_connectedCells_1"
        FOREIGN KEY ("connectedCells_1") REFERENCES "CellSpace" ("CellSpaceID"),
    CONSTRAINT "FK_InterLayerConnection_connectedCells_2"
        FOREIGN KEY ("connectedCells_2") REFERENCES "CellSpace" ("CellSpaceID"),
    CONSTRAINT "chk_InterLayerConnection_connectedNodes_pair" CHECK (
        ("connectedNodes_1" IS NULL AND "connectedNodes_2" IS NULL)
        OR ("connectedNodes_1" IS NOT NULL AND "connectedNodes_2" IS NOT NULL)
    ),
    CONSTRAINT "chk_InterLayerConnection_connectedCells_pair" CHECK (
        ("connectedCells_1" IS NULL AND "connectedCells_2" IS NULL)
        OR ("connectedCells_1" IS NOT NULL AND "connectedCells_2" IS NOT NULL)
    )
);

COMMENT ON TABLE "InterLayerConnection"
    IS 'Describes the connection between two thematic layers. ';

COMMENT ON COLUMN "InterLayerConnection"."typeOfTopoExpression"
    IS 'Describes the topological relationship between two layers (e.g.
    overlaps, contains, etc.).';
```

Listing 18 – A SQL schema of the InterLayerConnection table

The following examples refer to an InterLayerConnection row (“ILC-R1-1”), shown in Figure 3. For example, consider a cell space (“R1”) containing a step that pedestrians can traverse but that is difficult for wheelchairs to cross. If this distinction leads to the creation of separate thematic layers for pedestrians and wheelchairs, the cell space “R1” is split into “R1-1” and “R1-2” based on the step (a process known as subspacing). In this case, since “R1” and “R1-1” (and “R1-2”) represent the same physical space, they have a “Contains” relationship. As shown in the example, this can be represented as “ILC-R1-1” (and “ILC-R1-2”).

```
INSERT INTO "InterLayerConnection" (
    "InterLayerConnectionID", "IndoorFeaturesID", "typeOfTopoExpression",
    "connectedLayers_1", "connectedLayers_2",
    "connectedCells_1", "connectedCells_2",
    "connectedNodes_1", "connectedNodes_2"
) VALUES (
    'ILC-R1-1', 'IFs', 'Contains',
    'TL-Pedestrian', 'TL-Wheelchair',
    'R1', 'R1-1',
    'N-R1', 'N-R1-1'
);
```

Listing 19 – An example INSERT statement for the InterLayerConnection table

REQUIREMENT 11: INTERLAYERCONNECTION SCHEMA	
IDENTIFIER	/req/core/interlayerconnection
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
A	An InterLayerConnection table SHALL reference the two connected ThematicLayer rows using the connectedLayers_1 and connectedLayers_2 columns.
B	An InterLayerConnection table SHALL store the topological relationship in the typeOfTopoExpression column, whose subject is connectedLayers_1 (and, when present, connectedNodes_1 / connectedCells_1).
C	The typeOfTopoExpression value SHALL be one of “Contains”, “Overlaps”, “Equals”, “Within”, “Crosses”, or “Other”.
D	The values of connectedLayers_*, connectedNodes_*, and connectedCells_* SHALL be ordered and mapped to the same indices (_1 with _1, _2 with _2).
E	When connected nodes are provided, an InterLayerConnection table SHALL populate both connectedNodes_1 and connectedNodes_2.
F	When connected cell spaces are provided, an InterLayerConnection table SHALL populate both connectedCells_1 and connectedCells_2.

7.11. ExternalReferenceType

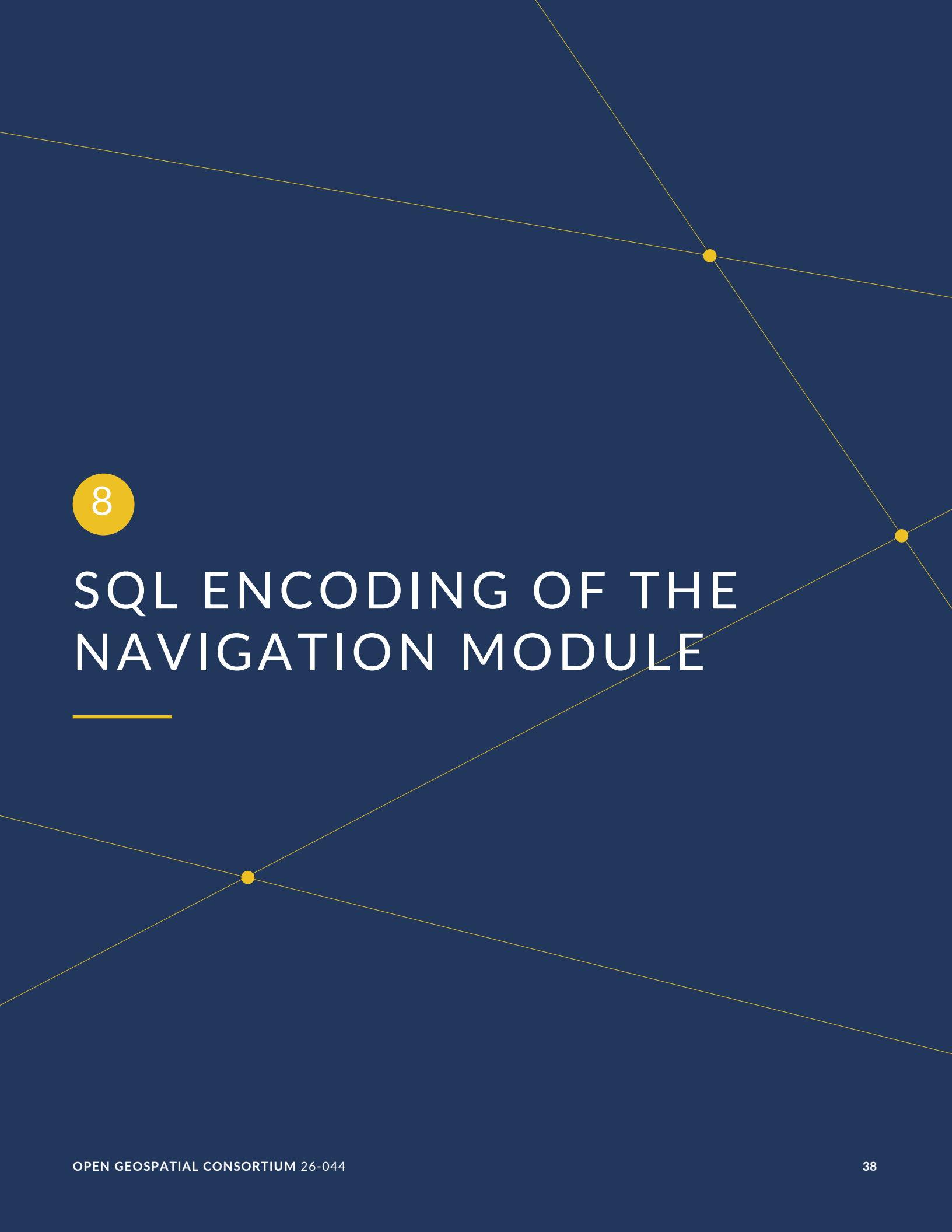
The UML DataTypes ExternalObjectReferenceType and ExternalReferenceType are encoded as PostgreSQL composite types rather than as tables. These are not feature classes of IndoorGML 2.0. CellSpace and CellBoundary optionally store an externalReference column of type ExternalReferenceType, which nests an externalObject of type ExternalObjectReferenceType together with an informationSystem URI.

```
/* UML DataType ExternalObjectReferenceType / ExternalReferenceType
   Encoded as PostgreSQL composite types (not tables).
   Apply before creating CellSpace / CellBoundary. */
```

```
CREATE TYPE "ExternalObjectReferenceType" AS (
    "name" varchar(100),
    "uri" text
);

CREATE TYPE "ExternalReferenceType" AS (
    "externalObject" "ExternalObjectReferenceType",
    "informationSystem" text
);
```

Listing 20 — A SQL schema of the ExternalReference composite types



8

SQL ENCODING OF THE NAVIGATION MODULE

SQL ENCODING OF THE NAVIGATION MODULE

REQUIREMENTS CLASS 2	
IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
TARGET TYPE	Implementation Specification
CONFORMANCE CLASS	Conformance class A.2: http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/navi
PREREQUISITES	http://www.opengis.net/spec/indoorgml/2.0/req Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
NORMATIVE STATEMENTS	Requirement 12: /req/navi/global Requirement 13: /req/navi/navigablespace Requirement 14: /req/navi/generalspace Requirement 15: /req/navi/transferspace Requirement 16: /req/navi/objectspace Requirement 17: /req/navi/navigableboundary Requirement 18: /req/navi/route

8.1. General requirements for encoding standards

The navigation SQL encoding schema extends the core SQL encoding schema with tables for navigation-related feature classes.

REQUIREMENT 12: REQUIREMENT FOR GLOBAL SQL SCHEMA OF THE NAVIGATION MODULE	
IDENTIFIER	/req/navi/global
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A database representing an IndoorGML 2.0 Navigation module dataset SHALL implement the SQL schema defined in this standard (including IndoorGML_navi.sql and IndoorGML_core.sql).

Navigation code list ENUM types used by the navigation module tables are specified in Annex D. An informative example CodeList derived from OmniClass is provided in Annex E. That CodeList does not extend the closed ENUM types of the reference schema.

8.2. NavigableSpace

The NavigableSpace table extends CellSpace with navigation-specific attributes.

The NavigableSpace table shall contain:

- NavigableSpaceID as a primary key and foreign key to CellSpace
- locomotionType of type LocomotionAccessType, with values including “Walking”, “Flying”, “Rolling”, and “Unspecified”

```
CREATE TABLE "NavigableSpace"
(
    "locomotionType" "LocomotionAccessType" NOT NULL,
    "NavigableSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NavigableSpace" PRIMARY KEY ("NavigableSpaceID"),
    CONSTRAINT "FK_NavigableSpace_CellSpace"
        FOREIGN KEY ("NavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID")
);
```

Listing 21 – A SQL schema of the NavigableSpace table

REQUIREMENT 13: NAVIGABLESPACE SCHEMA	
IDENTIFIER	/req/navi/navigablespace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A NavigableSpace table SHALL reference a CellSpace row and SHALL contain a locomotionType column.

8.3. GeneralSpace

The GeneralSpace table extends NavigableSpace with a function attribute.

The GeneralSpace table shall contain:

- GeneralSpaceID as a primary key and foreign key to NavigableSpace
- function of type GeneralSpaceFunctionType, with values including “Administration”, “Business, trade”, “Education, training”, “Recreation”, “Laboratory”, “Storage”, and “Security”

```
CREATE TABLE "GeneralSpace"
(
    "function" "GeneralSpaceFunctionType" NOT NULL,
    "GeneralSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_GeneralSpace" PRIMARY KEY ("GeneralSpaceID"),
    CONSTRAINT "FK_GeneralSpace_NavigableSpace"
        FOREIGN KEY ("GeneralSpaceID") REFERENCES "NavigableSpace"
        ("NavigableSpaceID")
);
```

Listing 22 – A SQL schema of the GeneralSpace table

REQUIREMENT 14: GENERALSPEACE SCHEMA

IDENTIFIER	/req/navi/generalspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A GeneralSpace table SHALL contain a function column.

8.4. TransferSpace

The TransferSpace table extends NavigableSpace with transfer-specific attributes.

The TransferSpace table shall contain:

- TransferSpaceID as a primary key and foreign key to NavigableSpace
- category of type TransferSpaceCategoryType, with values “Door” and “Window”
- function of type TransferSpaceFunctionType, with values “AnchorSpace” and “ConnectionSpace”

```
CREATE TABLE "TransferSpace"
(
    "function" "TransferSpaceFunctionType" NOT NULL,
    "category" "TransferSpaceCategoryType" NOT NULL,
    "TransferSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_TransferSpace" PRIMARY KEY ("TransferSpaceID"),
    CONSTRAINT "FK_TransferSpace_NavigableSpace"
        FOREIGN KEY ("TransferSpaceID") REFERENCES "NavigableSpace"
        ("NavigableSpaceID")
);
```

);

Listing 23 — A SQL schema of the TransferSpace table

REQUIREMENT 15: TRANSFERSPACE SCHEMA	
IDENTIFIER	/req/navi/transferspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A TransferSpace table SHALL contain category and function columns.

8.5. NonNavigableSpace

The NonNavigableSpace table extends CellSpace for spaces that are not navigable.

```
CREATE TABLE "NonNavigableSpace"
(
    "NonNavigableSpaceID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NonNavigableSpace" PRIMARY KEY ("NonNavigableSpaceID"),
    CONSTRAINT "FK_NonNavigableSpace_CellSpace"
        FOREIGN KEY ("NonNavigableSpaceID") REFERENCES "CellSpace"
        ("CellSpaceID")
);
```

Listing 24 — A SQL schema of the NonNavigableSpace table

8.6. ObjectSpace

The ObjectSpace table extends NonNavigableSpace with object-specific attributes.

The ObjectSpace table shall contain:

- ObjectSpaceID as a primary key and foreign key to NonNavigableSpace
- optionally, containedFeatures as an integer count of contained objects
- optionally, description

```
CREATE TABLE "ObjectSpace"
(
    "description" text NULL,
    "containedFeatures" integer NULL,
```

```

"ObjectSpaceID" varchar(100) NOT NULL,
CONSTRAINT "PK_ObjectSpace" PRIMARY KEY ("ObjectSpaceID"),
CONSTRAINT "FK_ObjectSpace_NonNavigableSpace"
FOREIGN KEY ("ObjectSpaceID") REFERENCES "NonNavigableSpace"
("NonNavigableSpaceID")
);

```

Listing 25 — A SQL schema of the ObjectSpace table

REQUIREMENT 16: OBJECTSPACE SCHEMA

IDENTIFIER	/req/navi/objectspace
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	An ObjectSpace table SHALL reference a NonNavigableSpace row.

8.7. NonNavigableBoundary

The NonNavigableBoundary table extends CellBoundary for boundaries that are not navigable.

```

CREATE TABLE "NonNavigableBoundary"
(
    "NonNavigableBoundaryID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NonNavigableBoundary" PRIMARY KEY ("NonNavigableBoundaryID"),
    CONSTRAINT "FK_NonNavigableBoundary_CellBoundary"
FOREIGN KEY ("NonNavigableBoundaryID") REFERENCES "CellBoundary"
("CellBoundaryID")
);

```

Listing 26 — A SQL schema of the NonNavigableBoundary table

8.8. NavigableBoundary

The NavigableBoundary table extends CellBoundary with navigation-specific boundary attributes.

The NavigableBoundary table shall contain:

- NavigableBoundaryID as a primary key and foreign key to CellBoundary
- optionally, boundaryOrientation

- NavigableBoundaryFunction of type NavigableBoundaryFunctionType, with values “AnchorBoundary” and “ConnectionBoundary”

```
CREATE TABLE "NavigableBoundary"
(
    "NavigableBoundaryFunction" "NavigableBoundaryFunctionType" NOT NULL,
    "boundaryOrientation" boolean NULL,
    "NavigableBoundaryID" varchar(100) NOT NULL,
    CONSTRAINT "PK_NavigableBoundary" PRIMARY KEY ("NavigableBoundaryID"),
    CONSTRAINT "FK_NavigableBoundary_CellBoundary"
        FOREIGN KEY ("NavigableBoundaryID") REFERENCES "CellBoundary"
        ("CellBoundaryID")
);
```

Listing 27 — A SQL schema of the NavigableBoundary table

REQUIREMENT 17: NAVIGABLEBOUNDARY SCHEMA

IDENTIFIER	/req/navi/navigableboundary
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A NavigableBoundary table SHALL contain a NavigableBoundaryFunction column.

8.9. Route

The Route table represents a navigation route as an ordered sequence of nodes and edges.

In this SQL encoding, routeNode and routeEdge are stored as jsonb arrays on the Route row. This design keeps the full traversal order in one place and makes it easier to maintain, reorder, or replace the node and edge sequence of a route without managing separate ordered junction-table rows. The array index corresponds to position in the route: routeNode[0] is the first node, routeNode[1] is the second node, and routeEdge follows the same rule for edges. Both routeNode and routeEdge SHALL be present and non-empty: routeNode SHALL contain at least two NodeID values, and routeEdge SHALL contain at least one EdgeID value.

The Route table shall contain:

- RouteID as the primary key
- optionally, creationDate
- routeNode as a jsonb array of ordered NodeID values (mandatory)
- routeEdge as a jsonb array of ordered EdgeID values (mandatory)

```
CREATE TABLE "Route"
(
    "creationDate" timestamp without time zone NULL,
    "RouteID" varchar(100) NOT NULL,
    "routeNode" jsonb NOT NULL,
    "routeEdge" jsonb NOT NULL,
    CONSTRAINT "PK_Route" PRIMARY KEY ("RouteID"),
    CONSTRAINT "chk_Route_nodes_and_edges" CHECK (
        jsonb_typeof("routeNode") = 'array'
        AND jsonb_typeof("routeEdge") = 'array'
        AND jsonb_array_length("routeNode") >= 2
        AND jsonb_array_length("routeEdge") >= 1
    )
);
```

Listing 28 – A SQL schema of the Route table

Requirement 18: Route Schema	
IDENTIFIER	/req/navi/route
INCLUDED IN	Requirements class 2: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi
A	A Route table SHALL store ordered node identifiers in the routeNode column as a JSON array of at least two NodeID values.
B	A Route table SHALL store ordered edge identifiers in the routeEdge column as a JSON array of at least one EdgeID value.
C	Both routeNode and routeEdge SHALL be present and non-empty on every Route row.



ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE



ANNEX A

(NORMATIVE)

CONFORMANCE CLASS ABSTRACT TEST SUITE

A.1. Conformance Class Core

CONFORMANCE CLASS A.1	
IDENTIFIER	http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/core
REQUIREMENTS CLASS	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/core
TARGET TYPE	Implementation Specification
CONFORMANCE TESTS	Abstract test A.1: /conf/core/global Abstract test A.2: /conf/core/indoorsql Abstract test A.3: /conf/core/indoorfeature Abstract test A.4: /conf/core/thematiclayer Abstract test A.5: /conf/core/primalspacelayer Abstract test A.6: /conf/core/cellspace Abstract test A.7: /conf/core/cellboundary Abstract test A.8: /conf/core/dualspacelayer Abstract test A.9: /conf/core/node Abstract test A.10: /conf/core/edge Abstract test A.11: /conf/core/interlayerconnection

ABSTRACT TEST A.1	
IDENTIFIER	/conf/core/global
REQUIREMENT	Requirement 1: /req/core/global

ABSTRACT TEST A.1

TEST PURPOSE Verify that a database implementing IndoorGML 2.0 Core implements the SQL schema defined in this standard (IndoorGML_core.sql), including the PostGIS geometry types specified in Clause 5.

TEST METHOD Inspect the database schema against IndoorGML_core.sql to verify the above requirement.

ABSTRACT TEST A.2

IDENTIFIER /conf/core/indoorsql

REQUIREMENT Requirement 2: /req/core/indoorsql

TEST PURPOSE Verify that each feature table of the SQL encoding contains a string primary key column for the feature identifier.

TEST METHOD Inspect the database schema to verify the above requirement.

ABSTRACT TEST A.3

IDENTIFIER /conf/core/indoorfeature

REQUIREMENT Requirement 3: /req/core/indoorfeature

TEST PURPOSE Verify that an IndoorFeatures table is related to at least one ThematicLayer row through a foreign key.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.4

IDENTIFIER /conf/core/thematiclayer

REQUIREMENT Requirement 4: /req/core/thematiclayer

TEST PURPOSE

For the following columns of the ThematicLayer table, verify that:

- The semanticExtension column declares whether there is an extension module with additional semantic information.
- The theme column determines what type of model representation can be expected.
- The theme value is one of “Virtual”, “Physical”, “Tags”, and “Unknown”.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.5

IDENTIFIER	/conf/core/primalspacelayer
REQUIREMENT	Requirement 5: /req/core/primalspacelayer
TEST PURPOSE	Verify that a PrimalSpaceLayer table is related to one or more CellSpace rows.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.6

IDENTIFIER	/conf/core/cellspace
REQUIREMENT	Requirement 6: /req/core/cellspace
TEST PURPOSE	For the following columns of the CellSpace table, verify that: • The PoI column declares whether it is a point of interest space. • The duality column references the NodeID of the corresponding Node. • Every non-null cellSpaceGeom_Geometry2D value has the geometry type POLYGON or MULTIPOLYGON. • Every non-null cellSpaceGeom_Geometry3D value has the geometry type POLYHEDRALSURFACE or TIN. • Every non-null cellSpaceGeom_Geometry3D value has three coordinate dimensions and satisfies CG_IsSolid(CG_MakeSolid(...)).
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.7

IDENTIFIER	/conf/core/cellboundary
REQUIREMENT	Requirement 7: /req/core/cellboundary
TEST PURPOSE	For the following columns of the CellBoundary table, verify that: • The isVirtual column specifies whether the boundary is virtual. • Every non-null cellBoundaryGeom_Geometry1D value has the geometry type LINESTRING or MULTILINESTRING. • Every non-null cellBoundaryGeom_Geometry2D value has the geometry type POLYGON or MULTIPOLYGON.

ABSTRACT TEST A.7

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.8

IDENTIFIER /conf/core/dualspacelayer

REQUIREMENT Requirement 8: /req/core/dualspacelayer

TEST PURPOSE For the following columns and associations of the DualSpaceLayer table, verify that:

- The isLogical column specifies whether the provided graph is geometric or logical.
- The isDirected column specifies whether the provided graph is directed.
- The DualSpaceLayer table is related to one or more Node rows.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.9

IDENTIFIER /conf/core/node

REQUIREMENT Requirement 9: /req/core/node

TEST PURPOSE For the following columns of the Node table, verify that:

- The duality column references the CellSpaceID of the corresponding CellSpace.
- Connected edge identifiers are stored in the connects column.
- Every non-null geometry value has the geometry type POINT.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.10

IDENTIFIER /conf/core/edge

REQUIREMENT Requirement 10: /req/core/edge

TEST PURPOSE For the following columns of the Edge table, verify that:

- Connected Node rows are referenced using the connects_node1 and connects_node2 columns.
- A weight column is present.

ABSTRACT TEST A.10

- Every non-null geometry value has the geometry type LINESTRING or MULTILINESTRING.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.11

IDENTIFIER /conf/core/interlayerconnection

REQUIREMENT Requirement 11: /req/core/interlayerconnection

TEST PURPOSE For the following columns of the InterLayerConnection table, verify that:

- The connectedLayers_1 and connectedLayers_2 columns reference the two connected ThematicLayer rows.
- The typeOfTopoExpression column stores the topological relationship, with subject on the _1 side.
- The typeOfTopoExpression value is one of "Contains", "Overlaps", "Equals", "Within", "Crosses", or "Other".
- The paired connectedLayers_*, connectedNodes_*, and connectedCells_* columns are ordered and mapped to the same indices.
- When connected nodes are provided, both connectedNodes_1 and connectedNodes_2 are populated.
- When connected cell spaces are provided, both connectedCells_1 and connectedCells_2 are populated.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

A.2. Conformance Class Navigation

CONFORMANCE CLASS A.2

IDENTIFIER <http://www.opengis.net/spec/indoorgml-2/2.0/conf/sql/navi>

REQUIREMENTS CLASS Requirements class 2: <http://www.opengis.net/spec/indoorgml-2/2.0/sql/req/navi>

TARGET TYPE Implementation Specification

CONFORMANCE TESTS Abstract test A.12: /conf/navi/global
Abstract test A.13: /conf/navi/navigablespace

CONFORMANCE CLASS A.2

Abstract test A.14: /conf/navi/generalspace
Abstract test A.15: /conf/navi/transferspace
Abstract test A.16: /conf/navi/objectspace
Abstract test A.17: /conf/navi/navigableboundary
Abstract test A.18: /conf/navi/route

ABSTRACT TEST A.12

IDENTIFIER /conf/navi/global

REQUIREMENT Requirement 12: /req/navi/global

TEST PURPOSE Verify that a database implementing the IndoorGML 2.0 Navigation module implements the SQL schema defined in this standard (IndoorGML_navi.sql and IndoorGML_core.sql).

TEST METHOD Inspect the database schema against IndoorGML_navi.sql and IndoorGML_core.sql to verify the above requirement.

ABSTRACT TEST A.13

IDENTIFIER /conf/navi/navigablespace

REQUIREMENT Requirement 13: /req/navi/navigablespace

TEST PURPOSE Verify that a NavigableSpace table references a CellSpace row and contains a locomotionType column.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.14

IDENTIFIER /conf/navi/generalspace

REQUIREMENT Requirement 14: /req/navi/generalspace

TEST PURPOSE Verify that a GeneralSpace table contains a function column.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.15

IDENTIFIER	/conf/navi/transferspace
REQUIREMENT	Requirement 15: /req/navi/transferspace
TEST PURPOSE	Verify that a TransferSpace table contains category and function columns.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.16

IDENTIFIER	/conf/navi/objectspace
REQUIREMENT	Requirement 16: /req/navi/objectspace
TEST PURPOSE	Verify that an ObjectSpace table references a NonNavigableSpace row.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.17

IDENTIFIER	/conf/navi/navigableboundary
REQUIREMENT	Requirement 17: /req/navi/navigableboundary
TEST PURPOSE	Verify that a NavigableBoundary table contains a NavigableBoundaryFunction column.
TEST METHOD	Inspect the database schema and instance data to verify the above requirement.

ABSTRACT TEST A.18

IDENTIFIER	/conf/navi/route
REQUIREMENT	Requirement 18: /req/navi/route
TEST PURPOSE	For the following columns of the Route table, verify that: Ordered node identifiers are stored in the routeNode column as a JSON array of at least two values.

ABSTRACT TEST A.18

- Ordered edge identifiers are stored in the `routeEdge` column as a JSON array of at least one value.
- Both `routeNode` and `routeEdge` are present and non-empty.

TEST METHOD Inspect the database schema and instance data to verify the above requirement.



ANNEX B (NORMATIVE) SCHEMAS

B

ANNEX B (NORMATIVE) SCHEMAS

NOTE: The DDL listings in this annex are expressed for the PostgreSQL/PostGIS reference platform. Implementations on other SQL platforms (for example, Oracle Spatial) MAY apply an equivalent mapping that preserves the logical table structure, keys, associations, and cardinalities defined by this Standard (see Clause 5).

B.1. SQL Schema of IndoorGML Core

```
/* ----- */
/* Generated by Enterprise Architect Version 14.1 */
/* Created On : 07-Jul-2025 11:10:23 */
/* DBMS : PostgreSQL */
/* ----- */

CREATE EXTENSION IF NOT EXISTS postgis
;
CREATE EXTENSION IF NOT EXISTS postgis_sfcgal
;
/* Drop Tables */

DROP TABLE IF EXISTS "CellBoundary" CASCADE
;

DROP TABLE IF EXISTS "CellBoundaryGeometryType" CASCADE
;

DROP TABLE IF EXISTS "CellSpace" CASCADE
;

DROP TABLE IF EXISTS "CellSpaceGeometryType" CASCADE
;

DROP TABLE IF EXISTS "DualSpaceLayer" CASCADE
;

DROP TABLE IF EXISTS "Edge" CASCADE
;

DROP TABLE IF EXISTS "IndoorFeatures" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection" CASCADE
;
```



```

DROP TABLE IF EXISTS "InterLayerConnection_CellSpace" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection_Node" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection_ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection CellSpace" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection Node" CASCADE
;

DROP TABLE IF EXISTS "InterLayerConnection ThematicLayer" CASCADE
;

DROP TABLE IF EXISTS "Node" CASCADE
;

DROP TABLE IF EXISTS "PrimalSpaceLayer" CASCADE
;

DROP TABLE IF EXISTS "ThematicLayer" CASCADE
;

/* Drop ENUM and composite Types */

DROP TYPE IF EXISTS "TopoExpressionValue" CASCADE
;

DROP TYPE IF EXISTS "ThemeLayerValue" CASCADE
;

DROP TYPE IF EXISTS "ExternalReferenceType" CASCADE
;

DROP TYPE IF EXISTS "ExternalObjectReferenceType" CASCADE
;

/* Create ENUM Types */

CREATE TYPE "ThemeLayerValue" AS ENUM (
    'Virtual',
    'Physical',
    'Tags',
    'Unknown'
)
;

CREATE TYPE "TopoExpressionValue" AS ENUM (
    'Contains',
    'Overlaps',
    'Equals',
    'Within',
    'Crosses',
    'Other'
)
;

/* Create composite Types (UML DataTypes for external references) */

```

```

CREATE TYPE "ExternalObjectReferenceType" AS (
  "name" varchar(100),
  "uri" text
)
;

CREATE TYPE "ExternalReferenceType" AS (
  "externalObject" "ExternalObjectReferenceType",
  "informationSystem" text
)
;

/* Create Tables */

CREATE TABLE "CellBoundary"
(
  "cellBoundaryGeom_Geometry1D" geometry NULL,
  "cellBoundaryGeom_Geometry2D" geometry NULL,
  "externalReference" "ExternalReferenceType" NULL,
  "isVirtual" boolean NOT NULL,
  "CellBoundaryID" varchar(100) NOT NULL,
  "boundedBy" varchar(100) NULL,
  "PrimalSpaceLayerID" varchar(100) NOT NULL,
  CONSTRAINT "chk_CellBoundary_geom1d_curve" CHECK (
    "cellBoundaryGeom_Geometry1D" IS NULL
    OR GeometryType("cellBoundaryGeom_Geometry1D") IN ('LINESTRING',
'MULTILINESTRING')
  ),
  CONSTRAINT "chk_CellBoundary_geom2d_surface" CHECK (
    "cellBoundaryGeom_Geometry2D" IS NULL
    OR GeometryType("cellBoundaryGeom_Geometry2D") IN ('POLYGON', 'MULTIPOLYGON')
  )
)
;

CREATE TABLE "CellSpace"
(
  "cellSpaceGeom_Geometry2D" geometry NULL,
  "cellSpaceGeom_Geometry3D" geometry NULL,
  "cellSpaceName" varchar(100) NULL,
  "externalReference" "ExternalReferenceType" NULL,
  "level" varchar(100) NULL,
  "PoI" boolean NOT NULL,
  "CellSpaceID" varchar(100) NOT NULL,
  "PrimalSpaceLayerID" varchar(100) NOT NULL,
  "duality" varchar(100) NOT NULL,
  CONSTRAINT "chk_CellSpace_geom2d_surface" CHECK (
    "cellSpaceGeom_Geometry2D" IS NULL
    OR GeometryType("cellSpaceGeom_Geometry2D") IN ('POLYGON', 'MULTIPOLYGON')
  ),
  CONSTRAINT "chk_CellSpace_geom3d_solid" CHECK (
    "cellSpaceGeom_Geometry3D" IS NULL
    OR (
      GeometryType("cellSpaceGeom_Geometry3D") IN ('POLYHEDRALSURFACE', 'TIN')
      AND ST_NDims("cellSpaceGeom_Geometry3D") = 3
      AND CG_IsSolid(CG_MakeSolid("cellSpaceGeom_Geometry3D"))
    )
  )
)
;

CREATE TABLE "DualSpaceLayer"

```

```

(
  "creationDate" timestamp without time zone NULL,
  "terminationDate" timestamp without time zone NULL,
  "isLogical" boolean NOT NULL,
  "isDirected" boolean NOT NULL,
  "DualSpaceLayerID" varchar(100) NOT NULL,
  "ThematicLayerID" varchar(100) NOT NULL
)
;

CREATE TABLE "Edge"
(
  "geometry" geometry NULL,
  "weight" real NOT NULL,
  "EdgeID" varchar(100) NOT NULL,
  "duality" varchar(100) NULL,
  "DualSpaceLayerID" varchar(100) NOT NULL,
  connects_node1 varchar(100) NOT NULL,
  connects_node2 varchar(100) NOT NULL,
  CONSTRAINT "chk_Edge_geometry_curve" CHECK (
    "geometry" IS NULL
    OR GeometryType("geometry") IN ('LINESTRING', 'MULTILINESTRING')
  )
)
;

CREATE TABLE "IndoorFeatures"
(
  "IndoorFeaturesID" varchar(100) NOT NULL,
  "layers" varchar(100) NULL -- Reference to layers if needed, though usually
layers point back to IndoorFeatures
)
;

CREATE TABLE "InterLayerConnection"
(
  "typeOfTopoExpression" "TopoExpressionValue" NOT NULL,
  "comment" text NULL,
  "InterLayerConnectionID" varchar(100) NOT NULL,
  "IndoorFeaturesID" varchar(100) NULL,
  "connectedLayers_1" varchar(100) NOT NULL,
  "connectedLayers_2" varchar(100) NOT NULL,
  "connectedNodes_1" varchar(100) NULL,
  "connectedNodes_2" varchar(100) NULL,
  "connectedCells_1" varchar(100) NULL,
  "connectedCells_2" varchar(100) NULL,
  CONSTRAINT "chk_InterLayerConnection_connectedNodes_pair" CHECK (
    ("connectedNodes_1" IS NULL AND "connectedNodes_2" IS NULL)
    OR ("connectedNodes_1" IS NOT NULL AND "connectedNodes_2" IS NOT NULL)
  ),
  CONSTRAINT "chk_InterLayerConnection_connectedCells_pair" CHECK (
    ("connectedCells_1" IS NULL AND "connectedCells_2" IS NULL)
    OR ("connectedCells_1" IS NOT NULL AND "connectedCells_2" IS NOT NULL)
  )
)
;

CREATE TABLE "Node"
(
  "geometry" geometry NULL,
  "NodeID" varchar(100) NOT NULL,
  "duality" varchar(100) NOT NULL,
  "DualSpaceLayerID" varchar(100) NOT NULL,

```

```

connects jsonb NULL,
CONSTRAINT "chk_Node_geometry_point" CHECK (
    "geometry" IS NULL
    OR GeometryType("geometry") = 'POINT'
)
)
;

CREATE TABLE "PrimalSpaceLayer"
(
    "creationDate" timestamp without time zone NULL,
    "terminationDate" timestamp without time zone NULL,
    "PrimalSpaceLayerID" varchar(100) NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL
)
;

CREATE TABLE "ThematicLayer"
(
    "semanticExtension" boolean NOT NULL,
    "theme" "ThemeLayerValue" NOT NULL,
    "ThematicLayerID" varchar(100) NOT NULL,
    "IndoorFeaturesID" varchar(100) NOT NULL
)
;

/* Create Primary Keys, Indexes, Uniques, Checks */

ALTER TABLE "CellBoundary" ADD CONSTRAINT "PK_CellBoundary"
PRIMARY KEY ("CellBoundaryID")
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "PK_CellSpace"
PRIMARY KEY ("CellSpaceID")
;

ALTER TABLE "DualSpaceLayer" ADD CONSTRAINT "PK_DualSpaceLayer"
PRIMARY KEY ("DualSpaceLayerID")
;

ALTER TABLE "Edge" ADD CONSTRAINT "PK_Edge"
PRIMARY KEY ("EdgeID")
;

ALTER TABLE "IndoorFeatures" ADD CONSTRAINT "PK_IndoorFeatures"
PRIMARY KEY ("IndoorFeaturesID")
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "PK_InterLayerConnection"
PRIMARY KEY ("InterLayerConnectionID")
;

ALTER TABLE "Node" ADD CONSTRAINT "PK_Node"
PRIMARY KEY ("NodeID")
;

ALTER TABLE "PrimalSpaceLayer" ADD CONSTRAINT "PK_PrimalSpaceLayer"
PRIMARY KEY ("PrimalSpaceLayerID")
;

ALTER TABLE "ThematicLayer" ADD CONSTRAINT "PK_ThematicLayer"
PRIMARY KEY ("ThematicLayerID")
;

```

```

/* Create Foreign Key Constraints */

ALTER TABLE "CellBoundary" ADD CONSTRAINT "FK_CellBoundary_boundedBy"
FOREIGN KEY ("boundedBy") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "CellBoundary" ADD CONSTRAINT "FK_CellBoundary_cellBoundaryMember"
FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_cellSpaceMember"
FOREIGN KEY ("PrimalSpaceLayerID") REFERENCES "PrimalSpaceLayer"
("PrimalSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "CellSpace" ADD CONSTRAINT "FK_CellSpace_duality"
FOREIGN KEY ("duality") REFERENCES "Node" ("NodeID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "DualSpaceLayer" ADD CONSTRAINT "FK_DualSpaceLayer_dualSpace"
FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_duality"
FOREIGN KEY ("duality") REFERENCES "CellBoundary" ("CellBoundaryID") ON DELETE
No Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_DualSpaceLayer"
FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
("DualSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_connects_node1"
FOREIGN KEY (connects_node1) REFERENCES "Node" ("NodeID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "Edge" ADD CONSTRAINT "FK_Edge_connects_node2"
FOREIGN KEY (connects_node2) REFERENCES "Node" ("NodeID") ON DELETE No Action ON
UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
layerConnections"
FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
("IndoorFeaturesID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
connectedLayers_1"
FOREIGN KEY ("connectedLayers_1") REFERENCES "ThematicLayer"
("ThematicLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
connectedLayers_2"

```

```

FOREIGN KEY ("connectedLayers_2") REFERENCES "ThematicLayer"
("ThematicLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
connectedNodes_1"
FOREIGN KEY ("connectedNodes_1") REFERENCES "Node" ("NodeID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
connectedNodes_2"
FOREIGN KEY ("connectedNodes_2") REFERENCES "Node" ("NodeID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
connectedCells_1"
FOREIGN KEY ("connectedCells_1") REFERENCES "CellSpace" ("CellSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "InterLayerConnection" ADD CONSTRAINT "FK_InterLayerConnection_
connectedCells_2"
FOREIGN KEY ("connectedCells_2") REFERENCES "CellSpace" ("CellSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "Node" ADD CONSTRAINT "FK_Node_duality"
FOREIGN KEY ("duality") REFERENCES "CellSpace" ("CellSpaceID") ON DELETE No
Action ON UPDATE No Action
;

ALTER TABLE "Node" ADD CONSTRAINT "FK_Node_DualSpaceLayer"
FOREIGN KEY ("DualSpaceLayerID") REFERENCES "DualSpaceLayer"
("DualSpaceLayerID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "PrimalSpaceLayer" ADD CONSTRAINT "FK_PrimalSpaceLayer_primalSpace"
FOREIGN KEY ("ThematicLayerID") REFERENCES "ThematicLayer" ("ThematicLayerID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "ThematicLayer" ADD CONSTRAINT "FK_ThematicLayer_layers"
FOREIGN KEY ("IndoorFeaturesID") REFERENCES "IndoorFeatures"
("IndoorFeaturesID") ON DELETE No Action ON UPDATE No Action
;

/* Create Table Comments, Sequences for Autonumber Columns */

COMMENT ON TABLE "InterLayerConnection"
IS 'Describes the connection between two thematic layers. '
;

COMMENT ON COLUMN "InterLayerConnection"."typeOfTopoExpression"
IS 'Describes the topological relationship between two layers (e.g. overlaps,
contains, etc.).'
;

COMMENT ON TABLE "ThematicLayer"
IS 'A layer of specific theme aggregating a primal and/or a dual space of a
given environment.'
;

```

```

COMMENT ON COLUMN "ThematicLayer"."semanticExtension"
  IS 'Indicates whether semantic information is associated (true) or not (false)
  to the primal space of the thematic layer.'
;

COMMENT ON COLUMN "ThematicLayer"."theme"
  IS 'Determines the theme of the layer (e.g topographic, logical, etc.).'
;

```

Listing B.1

B.2. SQL Schema of IndoorGML Navigation

```

/* ----- */
/* IndoorGML Navigation Schema Extension */
/* Apply AFTER IndoorGML_core.sql on the same database */
/* ----- */

/* Drop Navigation Tables */

DROP TABLE IF EXISTS "TransferSpace" CASCADE
;

DROP TABLE IF EXISTS "GeneralSpace" CASCADE
;

DROP TABLE IF EXISTS "ObjectSpace" CASCADE
;

DROP TABLE IF EXISTS "NonNavigableSpace" CASCADE
;

DROP TABLE IF EXISTS "NavigableBoundary" CASCADE
;

DROP TABLE IF EXISTS "NonNavigableBoundary" CASCADE
;

DROP TABLE IF EXISTS "NavigableSpace" CASCADE
;

DROP TABLE IF EXISTS "Route" CASCADE
;

DROP TABLE IF EXISTS "GeneralSpaceFunctionType" CASCADE
;

DROP TABLE IF EXISTS "LocomotionAccessType" CASCADE
;

DROP TABLE IF EXISTS "NavigableBoundaryFunctionType" CASCADE
;

DROP TABLE IF EXISTS "NetworkType" CASCADE
;

```

```

DROP TABLE IF EXISTS "TransferSpaceFunctionType" CASCADE
;

DROP TABLE IF EXISTS "TransferSpaceType" CASCADE
;

/* Drop ENUM Types */

DROP TYPE IF EXISTS "NavigableBoundaryFunctionType" CASCADE
;

DROP TYPE IF EXISTS "GeneralSpaceFunctionType" CASCADE
;

DROP TYPE IF EXISTS "LocomotionAccessType" CASCADE
;

DROP TYPE IF EXISTS "TransferSpaceCategoryType" CASCADE
;

DROP TYPE IF EXISTS "TransferSpaceFunctionType" CASCADE
;

/* Create ENUM Types */

CREATE TYPE "GeneralSpaceFunctionType" AS ENUM (
    'Administration',
    'Business, trade',
    'Education, training',
    'Recreation',
    'Laboratory',
    'Storage',
    'Security'
)
;

CREATE TYPE "LocomotionAccessType" AS ENUM (
    'Walking',
    'Flying',
    'Rolling',
    'Unspecified'
)
;

CREATE TYPE "NavigableBoundaryFunctionType" AS ENUM (
    'AnchorBoundary',
    'ConnectionBoundary'
)
;

CREATE TYPE "TransferSpaceFunctionType" AS ENUM (
    'ConnectionSpace',
    'AnchorSpace'
)
;

CREATE TYPE "TransferSpaceCategoryType" AS ENUM (
    'Door',
    'Window'
)
;

/* Create Navigation Tables */

```



```

CREATE TABLE "NavigableSpace"
(
    "locomotionType" "LocomotionAccessType" NOT NULL,
    "NavigableSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "NonNavigableSpace"
(
    "NonNavigableSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "GeneralSpace"
(
    "function" "GeneralSpaceFunctionType" NOT NULL,
    "GeneralSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "TransferSpace"
(
    "function" "TransferSpaceFunctionType" NOT NULL,
    "category" "TransferSpaceCategoryType" NOT NULL,
    "TransferSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "ObjectSpace"
(
    "description" text NULL,
    "containedFeatures" integer NULL,
    "ObjectSpaceID" varchar(100) NOT NULL
)
;

CREATE TABLE "NavigableBoundary"
(
    "NavigableBoundaryFunction" "NavigableBoundaryFunctionType" NOT NULL,
    "boundaryOrientation" boolean NULL,
    "NavigableBoundaryID" varchar(100) NOT NULL
)
;

CREATE TABLE "NonNavigableBoundary"
(
    "NonNavigableBoundaryID" varchar(100) NOT NULL
)
;

CREATE TABLE "Route"
(
    "creationDate" timestamp without time zone NULL,
    "RouteID" varchar(100) NOT NULL,
    "routeNode" jsonb NOT NULL,
    "routeEdge" jsonb NOT NULL,
    CONSTRAINT "chk_Route_nodes_and_edges" CHECK (
        jsonb_typeof("routeNode") = 'array'
        AND jsonb_typeof("routeEdge") = 'array'
        AND jsonb_array_length("routeNode") >= 2
        AND jsonb_array_length("routeEdge") >= 1
    )
)

```

```

)
;

/* Primary Keys */

ALTER TABLE "NavigableSpace" ADD CONSTRAINT "PK_NavigableSpace"
PRIMARY KEY ("NavigableSpaceID")
;

ALTER TABLE "NonNavigableSpace" ADD CONSTRAINT "PK_NonNavigableSpace"
PRIMARY KEY ("NonNavigableSpaceID")
;

ALTER TABLE "GeneralSpace" ADD CONSTRAINT "PK_GeneralSpace"
PRIMARY KEY ("GeneralSpaceID")
;

ALTER TABLE "TransferSpace" ADD CONSTRAINT "PK_TransferSpace"
PRIMARY KEY ("TransferSpaceID")
;

ALTER TABLE "ObjectSpace" ADD CONSTRAINT "PK_ObjectSpace"
PRIMARY KEY ("ObjectSpaceID")
;

ALTER TABLE "NavigableBoundary" ADD CONSTRAINT "PK_NavigableBoundary"
PRIMARY KEY ("NavigableBoundaryID")
;

ALTER TABLE "NonNavigableBoundary" ADD CONSTRAINT "PK_NonNavigableBoundary"
PRIMARY KEY ("NonNavigableBoundaryID")
;

ALTER TABLE "Route" ADD CONSTRAINT "PK_Route"
PRIMARY KEY ("RouteID")
;

/* Foreign Keys */

ALTER TABLE "NavigableSpace" ADD CONSTRAINT "FK_NavigableSpace_CellSpace"
FOREIGN KEY ("NavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "NonNavigableSpace" ADD CONSTRAINT "FK_NonNavigableSpace_CellSpace"
FOREIGN KEY ("NonNavigableSpaceID") REFERENCES "CellSpace" ("CellSpaceID") ON
DELETE No Action ON UPDATE No Action
;

ALTER TABLE "GeneralSpace" ADD CONSTRAINT "FK_GeneralSpace_NavigableSpace"
FOREIGN KEY ("GeneralSpaceID") REFERENCES "NavigableSpace" ("NavigableSpaceID")
ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "TransferSpace" ADD CONSTRAINT "FK_TransferSpace_NavigableSpace"
FOREIGN KEY ("TransferSpaceID") REFERENCES "NavigableSpace"
("NavigableSpaceID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "ObjectSpace" ADD CONSTRAINT "FK_ObjectSpace_NonNavigableSpace"
FOREIGN KEY ("ObjectSpaceID") REFERENCES "NonNavigableSpace"
("NonNavigableSpaceID") ON DELETE No Action ON UPDATE No Action
;

```

```

ALTER TABLE "NavigableBoundary" ADD CONSTRAINT "FK_NavigableBoundary_
CellBoundary"
FOREIGN KEY ("NavigableBoundaryID") REFERENCES "CellBoundary"
("CellBoundaryID") ON DELETE No Action ON UPDATE No Action
;

ALTER TABLE "NonNavigableBoundary" ADD CONSTRAINT "FK_NonNavigableBoundary_
CellBoundary"
FOREIGN KEY ("NonNavigableBoundaryID") REFERENCES "CellBoundary"
("CellBoundaryID") ON DELETE No Action ON UPDATE No Action
;

```

Listing B.2



ANNEX C (NORMATIVE) CORE CODE LIST TYPES



ANNEX C (NORMATIVE) CORE CODE LIST TYPES

The following PostgreSQL ENUM types represent IndoorGML core code lists:

- ThemeLayerValue for the ThematicLayer.theme column;
- TopoExpressionValue for the InterLayerConnection.typeOfTopoExpression column.

```
CREATE TYPE "ThemeLayerValue" AS ENUM (  
    'Virtual',  
    'Physical',  
    'Tags',  
    'Unknown'  
);  
  
CREATE TYPE "TopoExpressionValue" AS ENUM (  
    'Contains',  
    'Overlaps',  
    'Equals',  
    'Within',  
    'Crosses',  
    'Other'  
);
```

Listing C.1 — A SQL schema of the core code list types



ANNEX D (NORMATIVE) NAVIGATION CODE LIST TYPES

D

ANNEX D (NORMATIVE) NAVIGATION CODE LIST TYPES

The following PostgreSQL ENUM types represent IndoorGML navigation code lists:

- `LocomotionAccessType` for the `NavigableSpace.locomotionType` column;
- `GeneralSpaceFunctionType` for the `GeneralSpace.function` column;
- `TransferSpaceFunctionType` for the `TransferSpace.function` column;
- `TransferSpaceCategoryType` for the `TransferSpace.category` column;
- `NavigableBoundaryFunctionType` for the `NavigableBoundary.NavigableBoundaryFunction` column.

NOTE: Annex E lists OmniClass example codes for space function and category values. Those codes are not ENUM labels of the types in this annex.

```
CREATE TYPE "GeneralSpaceFunctionType" AS ENUM (  
    'Administration',  
    'Business, trade',  
    'Education, training',  
    'Recreation',  
    'Laboratory',  
    'Storage',  
    'Security'  
);  
  
CREATE TYPE "LocomotionAccessType" AS ENUM (  
    'Walking',  
    'Flying',  
    'Rolling',  
    'Unspecified'  
);  
  
CREATE TYPE "NavigableBoundaryFunctionType" AS ENUM (  
    'AnchorBoundary',  
    'ConnectionBoundary'  
);  
  
CREATE TYPE "TransferSpaceFunctionType" AS ENUM (  
    'ConnectionSpace',  
    'AnchorSpace'  
);  
  
CREATE TYPE "TransferSpaceCategoryType" AS ENUM (  
    'Door',  
    'Window'
```

);

Listing D.1 — A SQL schema of the navigation code list types



ANNEX E (INFORMATIVE) EXAMPLE OF CODELIST



ANNEX E (INFORMATIVE) EXAMPLE OF CODELIST

For mapping indoor space features to IndoorNavigation module classes, space features have to be classified by functions and usages of space. Because misspellings or different names for the same notion bring problems in data interoperability, this annex provides an example CodeList for space category types and space function types. The CodeLists are defined from OmniClass (Table 13 and Table 14) created and used by the North American architectural, engineering and construction (AEC) industry.

NOTE: The OmniClass codes in this annex (for example T1000, G1000, C1000) are informative examples.

E.1. GeneralSpace

E.1.1. GeneralSpaceFunctionType

Code list derived from OmniClass Table 13:

- T1000 – Corridor
- T1010 – Breezeway
- T1020 – Box Lobby
- T1030 – Elevator Lobby
- T1040 – Landing
- T1050 – Aisle
- T1060 – Ramp
- T1070 – Concourse
- T1080 – Moving walkway
- T1090 – Entry Lobby

- T1100 – Jet way
- T1110 – Elevator Shaft
- T1120 – Stair
- T1130 – Chute
- G1000 – Elevator Machine Room
- G1010 – Fire Command Center
- G1020 – Men's Restroom
- G1030 – Unisex Restroom
- G1040 – Refrigerant Machinery Room
- G1050 – Incinerator Room
- G1060 – Gas Room
- G1070 – Liquid Use, Dispensing and Mixing Room
- G1080 – Electrical Room
- G1090 – Telecommunications Room
- G1100 – Hazardous Waste Storage
- G1110 – Building Manager Office
- G1120 – Guard Stations
- G1130 – Women's Restroom
- G1140 – Furnace Room
- G1150 – Fuel Room
- G1160 – Liquid Storage Room
- G1170 – Hydrogen Cutoff Room
- G1180 – Switch Room
- G1190 – Classrooms
- G1200 – Assembly Hall
- G1210 – Physics Teaching Laboratory
- G1220 – Open Class Laboratory
- G1230 – Laboratory Service Space

- G1240 — Computer Lab
- G1250 — Training Support Space
- G1260 — Study Room
- G1270 — Evidence Room
- G1280 — Witness Stand
- G1290 — Robing Area
- G1300 — Council Chambers
- G1310 — Armory
- G1320 — General Performance Spaces
- G1330 — Orchestra Pit
- G1340 — Performance Hall
- G1350 — Audience Space
- G1360 — Audience Seating Space
- G1370 — Projection Booth
- G1380 — Stage Wings
- G1390 — Art Gallery
- G1400 — Sculpture Garden
- G1410 — Recording Studio
- G1420 — Photo Lab
- G1430 — Library
- G1440 — Mediation Chapel
- G1450 — Reflection Space
- G1460 — Chapel
- G1470 — Shrine
- G1480 — Confessional Space
- G1490 — Tabernacle
- G1500 — Choir Loft
- G1510 — Marriage Sanctuary

- G1520 – Mental Health Quiet Room
- G1530 – Bone Densitometry Room
- G1540 – CT Simulator Room
- G1550 – Head Radiographic Room
- G1560 – Mobile Imaging System Alcove
- G1570 – MRI System Component Room
- G1580 – PET/CT Simulator Room
- G1590 – Radiographic Room
- G1600 – Stereotactic Mammography Room
- G1610 – Ultrasound/Optical Coherence Tomography Room
- G1620 – Angiographic Control Room
- G1630 – Angiographic Procedure Control Area
- G1640 – Silver Collection Area
- G1650 – Computer Image Processing Area
- G1660 – CT Control Area
- G1670 – Image Quality Control Room
- G1680 – X-Ray, Plane Film Storage Space
- G1690 – MRI Control Room
- G1700 – MRI Viewing Room
- G1710 – Radiographic Control Room
- G1720 – Tele-Radiology/Tele-Medicine Room
- G1730 – Radiation Diagnostic and Therapy Spaces
- G1740 – Health Physics Laboratory
- G1750 – Linear Accelerator Entrance Maze, Healthcare
- G1760 – Radioactive Waste Storage Room, Healthcare
- G1770 – Nuclear Medicine Scanning Room
- G1780 – Patient Dose/Thyroid Uptake Room
- G1790 – Radiopharmacy

- G1800 – Radiation Therapy, Mold Fabrication Shop
- G1810 – Heart and Lung Diagnostic and Treatment Spaces
- G1820 – Cardiac Catheter Instrument Room
- G1830 – Cardiac Catheter Control Room
- G1840 – Cardiac Electrophysiology Room
- G1850 – Echocardiograph Room
- G1860 – Extended Pulmonary Function Testing Laboratory
- G1870 – Pacemaker ICD Interrogation Room
- G1880 – Procedure Viewing Area
- G1890 – Pulmonary Function Treadmill Room
- G1900 – Respiratory Therapy Clean-up Room
- G1910 – General Diagnostic Procedure and Treatment Spaces
- G1920 – Endoscopy/Gastroenterology Spaces
- G1930 – Clinical Laboratory Spaces
- G1940 – Pharmacy Spaces
- G1950 – Rehabilitation Spaces
- G1960 – Medical Research and Development Spaces
- G1970 – Chemistry Laboratories
- G1980 – Physical Sciences Laboratories
- G1990 – Earth and Environmental Sciences Laboratories
- G2000 – Psychology Laboratories
- G2010 – Dry Laboratories
- G2020 – Wet Laboratories
- G2030 – Biosciences Laboratories
- G2040 – Astronomy Laboratories
- G2050 – Forensics Laboratories
- G2060 – Bench Laboratories
- ... (Refer to the full standard implementation schema for other classifications)

E.2. TransferSpace

E.2.1. TransferSpaceCategoryType

- C1000 – Door
- C1010 – Vestibule
- C1020 – Sally port
- A1000 – Vestibule
- A1020 – Gate

E.2.2. TransferSpaceFunctionType

- C1000 – Door
- C1010 – Vestibule
- C1060 – Sally port
- A1000 – Entry Vestibule
- A1010 – Exterior door
- A1020 – Gate
- A1030 – Emergency door



ANNEX F (INFORMATIVE) REVISION HISTORY

F

ANNEX F (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-06-17	0.1	Zhiyong Wang	all	initial version derived from SQL generation workflow document
2026-06-18	0.2	Zhiyong Wang	5, 6, 7, 8, annex B	align schema with ENUM types, jsonb connects/route arrays, and EA table naming
2026-07-02	0.3	Zhiyong Wang	5, 6	add Part 2c-specific symbols and abbreviated terms table (clause 5); replace vague figure references with numbered cross-references in clause 6.1.1 and remove redundant follow-up sentences; rebuild document.pdf and document.html
2026-07-30	0.4	Zhiyong Wang	5, 6, 7	flatten geometry DataTypes to path-named PostGIS columns; encode ExternalReference DataTypes as Postgre SQL composite types; PascalCase table names; edition 0.0.9
2026-08-16	0.5	Zhiyong Wang	front matter, 5, 6, 7, 8, annex C	update document number to 26-044; replace Edge . connects jsonb with connects_node1 / connects_node2 foreign keys to Node; move Core code list types from Clause 7.1 to Annex C; define ObjectSpace . Containedfeatures as integer; edition 0.5.0
2026-08-17	0.5	Zhiyong Wang	abstract, 3, 4, 5, 6, 7, 8, annex A/B/C, schema	redefine Abstract Test Suite with one abstract test per requirement; align SQL attribute names and casing with UML; mark UML-mandatory attributes as NOT NULL; document GM_Solid encoding as POLYHEDRALSURFACE Z / TIN Z with geometry-type CHECK constraints; revise Abstract; add data type to Terms and Definitions and cite ISO 19103
2026-08-28	0.5	Zhiyong Wang	6, 8, annex D, annex E	add informative Annex E Example of CodeList (Omni Class); remove clause 6.1 Design Principle and promote former 6.1.1/6.1.2 to 6.1/6.2; retain encoding-principles sentence; shorten Annex E note



BIBLIOGRAPHY





BIBLIOGRAPHY

- [1] Butler H, Daly M, Doyle A, Gillies S, Hagen S, Schaub T, Internet Engineering Task Force: IETF RFC 7946, *The GeoJSON Format*. RFC Publisher (2016). <https://www.rfc-editor.org/info/rfc7946>.
- [2] Internet Engineering Task Force (committee): IETF RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). <https://www.rfc-editor.org/info/rfc8259>.
- [3] The PostGIS Development Group: **PostGIS Manual**, <https://postgis.net/docs/>
- [4] Ben-Kiki, O., Evans, C., Ingy döt Net: **YAML Ain't Markup Language**, <https://yaml.org/>